

RTL Design Sherpa

APB IOAPIC Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Ioapic Mas Index..... 13

2 ioapic..... 14

2.1 Overview..... 14

2.1.1 Introduction..... 14

2.1.2 Key Features..... 14

2.1.3 Applications..... 15

2.2 Functional Description..... 15

2.2.1 Design Philosophy..... 15

2.2.2 Comparison with 82093AA IOAPIC..... 16

2.2.3 82093AA Register Compatibility..... 17

2.3 Timing..... 17

2.3.1 Performance Characteristics..... 17

2.4 Waveforms..... 17

2.4.1 Waveform 1.1: Interrupt Delivery..... 17

2.4.2 Waveform 1.2: Redirection Table Write..... 18

2.4.3 Waveform 1.3: Level-Triggered Interrupt..... 18

2.4.4 Waveform 1.4: Interrupt Masking..... 19

2.5 Testing..... 19

2.5.1 Verification Status..... 19

2.5.2 Development Status..... 19

2.6 References..... 20

2.6.1 Related Documentation..... 20

2.7 Navigation....20
3 ioapic....22
3.1 Overview....22
3.1.1 High-Level Architecture....22
3.1.2 Figure 1.1: APB IOAPIC Block Diagram....22
3.2 Functional Description....22
3.2.1 Block Hierarchy....22
3.2.2 Data Flow....24
3.2.3 Indirect Access Method....24
3.2.4 Interrupt Flow State Machine....25
3.2.5 Figure 1.2: IOAPIC Delivery Stage....25
3.2.6 Address Space Organization....26
3.3 Timing....26
3.3.1 Clock Domain Architecture....26
3.3.2 Reset Architecture....28
3.4 Usage Example....29
3.4.1 Integration Guidelines....29
3.5 Design Notes....30
3.5.1 Design Trade-offs....30
3.6 Navigation....30
4 ioapic....31
4.1 Overview....31
4.2 Functional Description....31
4.2.1 Reset Signals....31

4.2.2 Reset Behavior....31
4.2.3 Reset Sequencing....32
4.3 Timing....32
4.3.1 Clock Domains....32
4.3.2 Clock Requirements....33
4.3.3 Interrupt Response Time....34
4.3.4 Timing Constraints....34
4.3.5 Clock Jitter and Stability....35
4.4 Design Notes....35
4.4.1 Clock Gating Considerations....35
4.5 Navigation....36
5 ioapic....37
5.1 Overview....37
5.2 References....37
5.2.1 Acronyms....37
5.2.2 Register Names....38
5.2.3 Key Terms....39
5.3 Navigation....39
6 ioapic....40
6.1 Overview....40
6.2 References....40
6.2.1 Primary Standards....40
6.2.2 Related RLB Documentation....40
6.2.3 External Resources....41

6.2.4 Design Methodology References....41
6.2.5 Historical Context....41
6.2.6 Software Examples and Drivers....42
6.2.7 Verification References....42
6.3 Navigation....42
7 ioapic....43
7.1 Overview....43
7.1.1 Module Hierarchy....43
7.1.2 Block Summary....43
7.1.3 Block Descriptions....44
7.2 Functional Description....44
7.2.1 Data Flow Between Blocks....44
7.2.2 Figure 2.1: IOAPIC Register Access Path....45
7.2.3 Interface Summary....45
7.3 Timing....46
7.3.1 Clock Domain Assignment....46
7.4 Related Modules....46
7.4.1 Module Dependencies....46
7.5 Navigation....46
8 ioapic....47
8.1 Overview....47
8.2 Functional Description....47
8.2.1 State Definitions....47
8.2.2 State Transitions....48

8.2.3 State Functions....49
8.2.4 Latched Signals....49
8.2.5 Edge vs Level Interrupt Paths....50
8.2.6 Arbitration Logic....50
8.2.7 Remote IRR Management....51
8.2.8 Multiple Pending IRQs....52
8.3 Navigation....53
9 ioapic....54
9.1 Overview....54
9.1.1 Register Access Method....54
9.2 Functional Description....54
9.2.1 Direct APB Registers....54
9.2.2 IOREGSEL Register (APB 0x000)....55
9.2.3 IOWIN Register (APB 0x004)....56
9.2.4 Internal Registers (Accessed via IOREGSEL/IOWIN)....56
9.2.5 Redirection Table (Internal Offsets 0x10-0x3F)....61
9.2.6 Complete Register Address Map....64
9.3 Usage Example....67
9.3.1 Example 1: Configure IRQ14 (IDE) - Edge-Triggered....67
9.3.2 Example 2: Configure IRQ9 (ACPI) - Level-Triggered....67
9.3.3 Example 3: Mask/Unmask an IRQ....68
9.3.4 Example 4: Check Delivery Status and Remote IRR....68
9.3.5 Example 5: Read IOAPIC Version and Capabilities....68
9.4 Design Notes....69

9.4.1 Register Field Summary....69
9.4.2 Reset Values....70
10 I/O Advanced Programmable Interrupt Controller (IOAPIC)....71
10.1 Overview....71
10.2 What is implemented....71
10.3 LowestPriority is delegated, and that is not a dodge....72
10.4 Arbitration is static unless you say otherwise....73
10.5 Destination mode....73
10.6 Feature history (RLB TASK-008 (was RLB-008), closed 2026-09-14)....73
10.7 Files....74
10.8 Verification....74
10.9 Specification....74
11 RLB (Retro Legacy Blocks) - FPGA Implementation Guide....76
11.1 What Are RLB (Retro Legacy Blocks)?....76
11.1.1 Overview....76
11.1.2 Purpose....76
11.2 Implemented RLB Modules....76
11.2.1 Complete Modules....76
11.3 FPGA Implementation - Nexys A7 / Genesys2....78
11.3.1 Can RLB Be Implemented on These Boards? YES!....78
11.3.2 Target FPGA Specifications....78
11.3.3 Resource Usage Estimates....78
11.3.4 What Can Fit on These Boards?....79
11.4 RLB System Integration on FPGA....80

11.4.1 Typical FPGA System Architecture....80
11.4.2 Example Use Cases on FPGAs....80
11.5 Nexys A7 Specific Integration....81
11.5.1 Available Resources on Nexys A7....81
11.5.2 How RLB Modules Map to Nexys A7....81
11.5.3 Example Nexys A7 Top-Level....81
11.6 Implementation Recommendations....82
11.6.1 For Nexys A7-100T....82
11.6.2 For Genesys 2....82
11.7 FPGA-Specific Considerations....83
11.7.1 Clock Management....83
11.7.2 Power Management....83
11.7.3 Interrupt Routing....83
11.7.4 External Interfaces....83
11.8 Soft Processor Options....84
11.8.1 MicroBlaze (Xilinx)....84
11.8.2 RISC-V (Open Source)....84
11.8.3 Custom Soft Core....84
11.9 Step-by-Step FPGA Implementation....84
11.9.1 Phase 1: Basic System (Week 1-2)....84
11.9.2 Phase 2: Expand Peripherals (Week 3-4)....85
11.9.3 Phase 3: Advanced Features (Week 5-6)....85
11.9.4 Phase 4: Application Development (Week 7+)....85
11.10 Example Address Map for FPGA....85

11.11 Software Development....86
11.11.1 Bare-Metal Firmware....86
11.11.2 Operating System Support....86
11.12 Advantages of RLB on FPGA....86
11.12.1 Development Benefits....86
11.12.2 Technical Benefits....87
11.12.3 Vs. Microcontroller....87
11.13 Conclusion....87
11.13.1 Can RLB Be Used on Nexys A7/Genesys2? ABSOLUTELY!....87
12 APB HPET Micro-Architecture Specification....89
12.1 Overview....89
12.2 Navigation....92
12.2.1 Chapter 1: Overview....92
12.2.2 Chapter 2: Blocks....92
12.2.3 Chapter 3: Interfaces....92
12.2.4 Chapter 4: Programming Model....92
12.2.5 Chapter 5: Registers....92

List of Figures

Figure 1.1: APB IOAPIC Block Diagram.....22
Figure 1.2: IOAPIC Delivery Stage.....25
Figure 2.1: IOAPIC Register Access Path.....45

List of Tables

Table 1: Table: Version History.....88

List of Waveforms

Waveform 1.1: Interrupt Delivery.....17
Waveform 1.2: Redirection Table Write.....18
Waveform 1.3: Level-Triggered Interrupt.....18
Waveform 1.4: Interrupt Masking.....19

1 Ioapic Mas Index

Generated: 2026-10-14

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 ioapic

2.1 Overview

2.1.1 Introduction

The APB I/O Advanced Programmable Interrupt Controller (IOAPIC) is the interrupt router you reach for when a plain 8259 won't cut it: 24 programmable interrupt inputs, flexible redirection to multiple CPUs, and both edge and level-triggered modes. Register access is 82093AA-compatible (the IOREGSEL/IOWIN indirect dance), and the bus side is a standard AMBA APB4 interface, so it drops into the RLB architecture without any special pleading.

2.1.2 Key Features

- **24 Independent IRQ Inputs**: IRQ0-IRQ23 with individual configuration per interrupt source
- **Programmable Redirection Table**: 64-bit entry per IRQ defining vector, mode, destination, trigger, polarity
- **Indirect Register Access**: 82093AA-compatible IOREGSEL/IOWIN mechanism for register access
- **Dual Trigger Modes**:
  - **Edge-triggered**: Latches interrupt on signal edge, delivered exactly once per edge - the pending latch clears on the CPU's accept, and a new edge landing in that same cycle sets it again rather than being lost (fixed 2026-09-09, issue #48)
  - **Level-triggered**: Tracks signal level, uses Remote IRR, requires EOI
- **Configurable Polarity**: Active-high or active-low per IRQ input
- **Priority Arbitration**: Static priority (lowest IRQ number wins, the 82093AA scheme and the reset default) or round robin behind `IOAPICARBCFG.rr_enable`
- **Delivery Modes**: Fixed mode (MVP), with support for LowestPri, SMI, NMI, INIT, ExtINT (future)
- **Remote IRR**: Level-triggered interrupt tracking with End-of-Interrupt (EOI) handling
- **APB Interface**: Standard AMBA APB4 compliant with 12-bit addressing
- **Clock Domain Crossing**: Optional CDC support via CDC_ENABLE parameter
- **PeakRDL Integration**: Register map generated from SystemRDL specification

- **82093AA Compatible**: Register layout and behavior match the 82093AA specification

2.1.3 Applications

Multi-Processor Systems: - Flexible interrupt routing to multiple CPUs - Per-IRQ destination configuration - Delivery mode selection per interrupt - Scalable interrupt distribution

PC-Compatible Systems: - x86 PC interrupt architecture - ISA bus interrupt routing - PCI interrupt support (INTA-INTD mapping) - Legacy IRQ redirection (IRQ0-15)

Embedded Systems: - Complex interrupt topologies - Priority-based interrupt handling - Mixed edge/level interrupt sources - Polarity-agnostic interrupt inputs

System Management: - Interrupt masking per source - Delivery status monitoring - Remote IRR tracking - Flexible interrupt mapping

2.2 Functional Description

2.2.1 Design Philosophy

Software Compatibility: The IOAPIC implements the 82093AA indirect register access method (IOREGSEL/IOWIN) for compatibility with existing software. This allows software written for legacy PC chipsets to work with minimal modifications.

Flexibility: Each of the 24 IRQ inputs can be independently configured for trigger mode (edge/level), polarity (active-high/low), delivery mode, destination CPU, and interrupt vector. This flexibility supports diverse system architectures.

Reliability: - 3-stage input synchronization prevents metastability - Edge detection on the synchronized inputs (no dedicated glitch filter: pulses shorter than the synchronizer depth are simply not seen) - Remote IRR prevents level interrupt re-triggering until EOI, per pin: a level interrupt that is never EOI'd blocks only its own input - EOI is matched against the vector that was actually delivered, so an RTE can be re-pointed while its interrupt is in flight - Delivery status tracking ensures reliable interrupt delivery

Standards Compliance: - **APB Protocol**: Full AMBA APB4 specification compliance - **PeakRDL**: Industry-standard SystemRDL for register generation - **82093AA**: Register layout and access method compatibility - **Reset Convention**: mixed – hand-written logic uses active-low asynchronous reset; the generated ioapic_regs.sv resets SYNCHRONOUSLY (active-high, derived from the reset input)

Modularity: Clean separation between interrupt routing logic (ioapic_core), register interface (ioapic_config_regs), and bus interface (apb4_ioapic) enables easy customization and integration.

2.2.2 Comparison with 82093AA IOAPIC

The APB IOAPIC draws directly from the 82093AA I/O APIC specification with RLB architecture enhancements:

Feature	82093AA	APB IOAPIC
Interface	Memory-mapped	AMBA APB4
Register Access	Indirect (IOREGSEL/IOWIN)	Indirect (IOREGSEL/IOWIN) — Same
IRQ Inputs	24 (IRQ0-23)	24 (IRQ0-23) — Same
Redirection Table	24 entries × 64-bit	24 entries × 64-bit — Same
Delivery Modes	Fixed, LowestPri, SMI, NMI, INIT, ExtINT	Fixed (MVP), others future
Destination Modes	Physical, Logical	Physical (MVP), Logical future
Trigger Modes	Edge, Level	Edge, Level — Same
Polarity	High, Low	High, Low — Same
Version Register	0x11, Max Entry 0x17	0x11, Max Entry 0x17 — Same
Remote IRR	Level interrupts	Level interrupts — Same
Priority	Implementation-defined	Static (lowest IRQ)
Multi-APIC	Supported	Future enhancement
Clock Domains	Single	Optional CDC support

Retained Features: - Indirect register access method - Redirection table structure - Edge/level trigger modes - Polarity configuration - Remote IRR mechanism - Delivery/destination fields

MVP Simplifications: - Fixed delivery mode only; the other modes are forwarded on `irq_out_deliv_mode` unmodified rather than acted on - Single IOAPIC in the block itself. Multi-IOAPIC routing is no longer “future”: `ioapic_deliv_merge` (RLB TASK-008, was RLB-008) merges N delivery channels outside the block, which is where it belongs – apb4_ioapic’s port list stays a single payload + valid/ready + status so a bridge can carry it onto a bus

RLB Enhancements: - Round-robin arbitration behind `IOAPICARBCFG.rr_enable`, at IOWIN selector 0x03. Not an 82093AA register: that selector is reserved on the real part, so a driver written for it never writes this and gets static priority - APB4 bus interface (instead of direct

memory-map) - Optional CDC for clock domain flexibility - PeakRDL register generation - Modern SystemVerilog coding practices - Comprehensive validation framework

2.2.3 82093AA Register Compatibility

Direct APB Registers: - `0x00`: IOREGSEL - Register offset selector - `0x04`: IOWIN - Data window for selected register - Nothing else in the 4 KB window is decoded: any other address, 0x08 upward, is dropped with PSLVERR (reads 0), so the indirect pair is the only way to the internal register file - 82093AA semantics, see Chapter 5

Internal Registers (via IOREGSEL/IOWIN): - **0x00**: IOAPICID - I/O APIC identification - **0x01**: IOAPICVER - Version (0x11) and Max Entry (0x17 for 24 IRQs) - **0x02**: IOAPICARB - Arbitration priority (read-only) - **0x10-0x3F**: IOREDTBL - Redirection table (24 entries × 2 registers)

Each redirection entry is 64 bits: - **LO register**: Vector, delivery mode, dest mode, polarity, trigger, mask, status fields - **HI register**: Destination CPU APIC ID

This matches the 82093AA specification exactly for software compatibility.

2.3 Timing

2.3.1 Performance Characteristics

Interrupt Latency: - IRQ detection: 3 clock cycles (synchronization) - Edge detection: 1 clock cycle - Arbitration: Combinational (<1 cycle) - Delivery initiation: 1 clock cycle - **Total:** ~5 clock cycles from IRQ assertion to delivery request

Register Access Performance: - Direct APB access (IOREGSEL): 2 APB clock cycles - Indirect access (IOWIN): 2 APB clock cycles per register - Full redirection entry (LO+HI): 4 transactions (~8 cycles) through IOREGSEL/IOWIN; there is no direct decode to shortcut it - With CDC: Add 2-4 cycles for synchronization

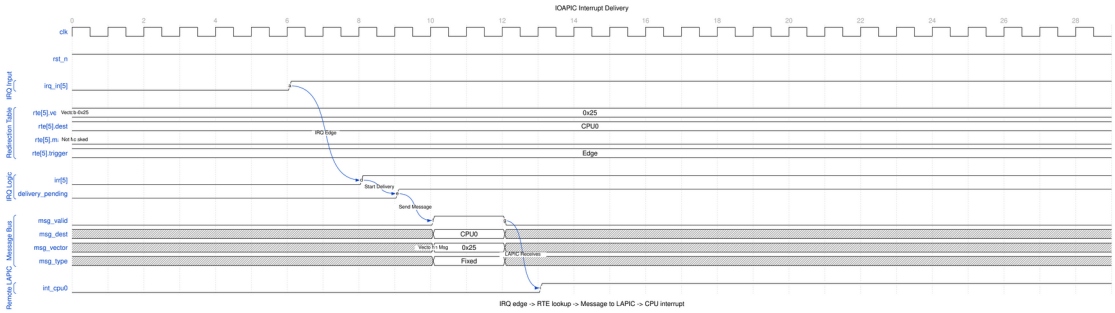
Resource Utilization (Post-Synthesis Estimates): - No CDC: ~800-1000 LUTs, ~600-800 flip-flops - With CDC: ~1000-1200 LUTs, ~800-1000 flip-flops - BRAM: None (all logic-based)

Scalability: Fixed 24 IRQ inputs per the 82093AA specification. For more IRQs, use multiple IOAPIC instances with different APIC IDs.

2.4 Waveforms

2.4.1 Waveform 1.1: Interrupt Delivery

Shows the flow from IRQ input to message delivery to LAPIC.

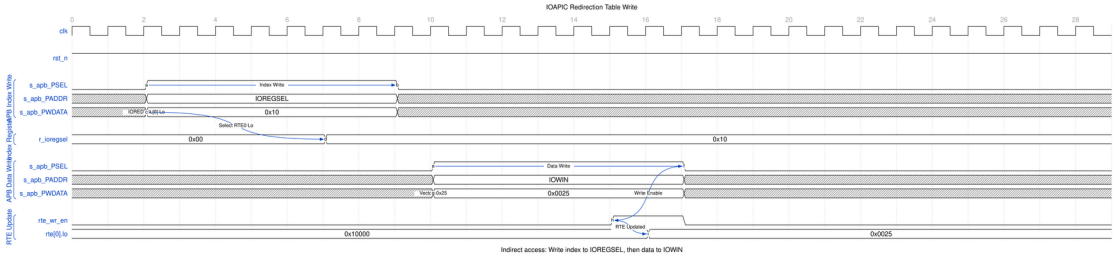


IOAPIC Interrupt Delivery

When an IRQ edge is detected, the corresponding IRR bit sets. The redirection table entry (RTE) is consulted for vector, destination, and delivery mode. An interrupt message is sent to the target LAPIC.

2.4.2 Waveform 1.2: Redirection Table Write

Indirect register access to configure an RTE.

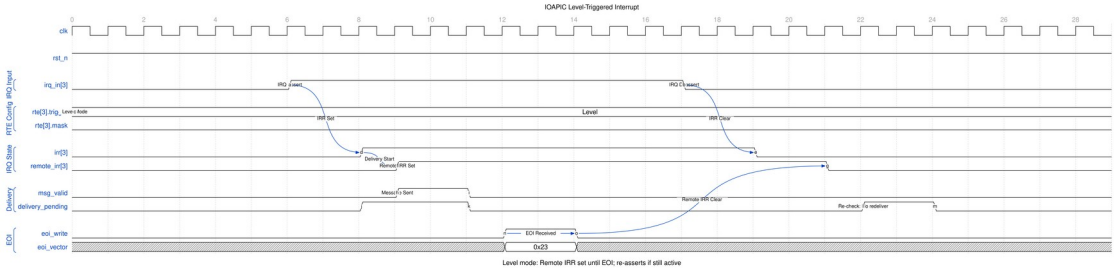


IOAPIC RTE Write

Two APB transactions required: 1. Write index to IOREGSEL (selects RTE low or high word) 2. Write data to IOWIN (updates the selected RTE)

2.4.3 Waveform 1.3: Level-Triggered Interrupt

Level mode with Remote IRR and EOI handling.

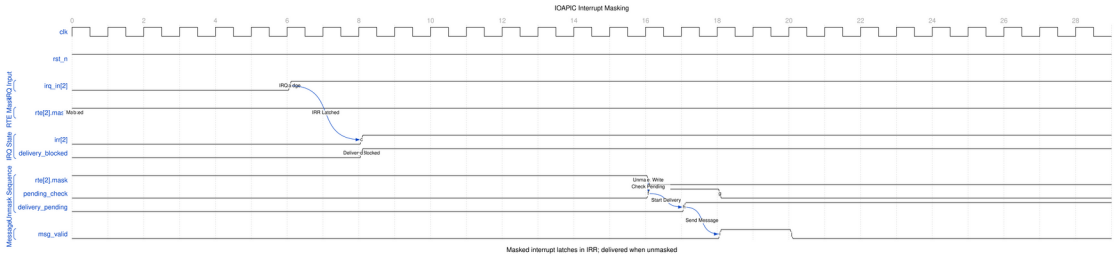


IOAPIC Level Triggered

For level-triggered interrupts: - Remote IRR set on delivery, blocking re-delivery - EOI clears Remote IRR on every pin delivered with that vector - If IRQ still asserted, re-delivery occurs

2.4.4 Waveform 1.4: Interrupt Masking

Masked interrupts latch in IRR and deliver when unmasked.



IOAPIC Mask Interrupt

When an IRQ arrives while masked, the IRR bit latches but delivery is blocked. Upon unmask, the IOAPIC checks for pending interrupts and delivers them.

2.5 Testing

2.5.1 Verification Status

Implementation Status: RTL Complete - 36/36 in all six DV configurations (CDC off/on x gate/func/full, 2026-09-09)

Completed Implementation: - [x] PeakRDL register specification with indirect access - [x] Core interrupt routing logic (edge/level/polarity) - [x] Priority arbitration (static) - [x] One-entry valid/ready delivery stage with per-pin EOI handling - [x] Remote IRR management - [x] Configuration register wrapper - [x] APB top-level with CDC support, LAPIC interface presented in pclk - [x] Complete filelist and documentation

Validation Coverage (`dv/tests/test_apb4_ioapic.py`): - [x] APB indirect register access tests - [x] Edge-triggered IRQ tests (all 24 inputs) - [x] Level-triggered IRQ tests with Remote IRR - [x] Polarity tests (active-high/low) - [x] Priority arbitration tests - [x] Delivery status tests - [x] EOI handling with level interrupts - [x] Redirection table configuration tests - [x] CDC mode validation - [x] Issue #48 defect regressions (`dv/tbclasses/ioapic/ioapic_tests_medium.py`): single delivery per edge, no parked valid, per-pin blocking, wrong-vector EOI, EOI before accept, mid-service vector rewrite, unmapped selector readback, no aliasing above 0x0FF, single-pclk EOI through the CDC

2.5.2 Development Status

Status: MVP Complete - issue #48 defects fixed 2026-09-09

MVP Scope Delivered: - [x] 24 IRQ inputs with synchronization - [x] Edge and level trigger detection - [x] Active high/low polarity support - [x] Fixed delivery mode - [x] Physical destination

mode - [x] Static priority arbitration, and round robin behind a control bit - [x] Remote IRR for level interrupts - [x] EOI handling - [x] Indirect register access (IOREGSEL/IOWIN) - [x] Complete redirection table - [x] Delivery status per IRQ - [x] Logical destination mode (`irq_out_dest_mode` carries the RTE's mode alongside the destination; the local APICs do the matching) - [x] Round-robin arbitration behind `IOAPICARBCFG.rr_enable`

Feature history (RLB TASK-008 (was RLB-008), closed 2026-09-14 in `vault/Tasks/RLB/closed.md`): - [x] LowestPriority delivery mode, delegated: the IOAPIC forwards the mode and the destination set, the local APICs arbitrate, and `irq_out_retry` carries a failed arbitration back so the interrupt is re-offered - [ ] Additional delivery modes acted on rather than forwarded (SMI, NMI, INIT, ExtINT) - [x] Multi-IOAPIC support, delegated: `ioapic_deliv_merge` merges N delivery channels onto one in round robin and tags each message with its source id, so an EOI routes back to the IOAPIC holding that pin's Remote IRR - [x] Boot interrupt delivery, delegated: `ioapic_boot_intx` reroutes a masked pin to its mapped legacy PIC input, gated on IOAPICBOOTINTX.enable (selector 0x07) AND that pin's RTE mask – both terms matter, since without the mask an actively delivered pin would be taken twice. This entry used to say BLOCKED because the delivery-mode field has no SIPI encoding; that was a category error. INIT-SIPI-SIPI is a local APIC's AP-startup IPI - [x] MSI/MSI-X, delegated: `ioapic_msi_emit` turns a delivery message into a posted write through an `apb4_master_stub`. The old "an APB slave has no initiator port" was true and beside the point – the companion does the emitting. Address and data are registers (selectors 0x04/0x05), and a refusal that lands too late to act on is counted in IOAPICMSIDROP

2.6 References

2.6.1 Related Documentation

- `../../rtl/ioapic/README.md` - Block summary and verification entry point
- `vault/Tasks/RLB/closed.md` - RLB TASK-008 (was RLB-008), the IOAPIC feature arc (closed)
- `../../rtl/ioapic/README.md` - Register generation guide
- `../../../../../../vault/Tasks/RLB/RLB_STATUS_AND_ROADMAP.md` - System-wide planning
- 82093AA I/O APIC Datasheet

2.7 Navigation

This specification document is organized as follows:

- **Chapter 1 (this chapter)**: Overview, features, applications, software compatibility

- **Chapter 2**: Detailed block specifications (ioapic_core, config_regs, PeakRDL integration)
- **Chapter 3**: Interface specifications (APB, indirect access, IRQ, EOI)
- **Chapter 4**: Programming model (initialization, redirection table, edge/level handling)
- **Chapter 5**: Register definitions (address map, indirect access, field descriptions)

Next: Chapter 1.2 - Architecture

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

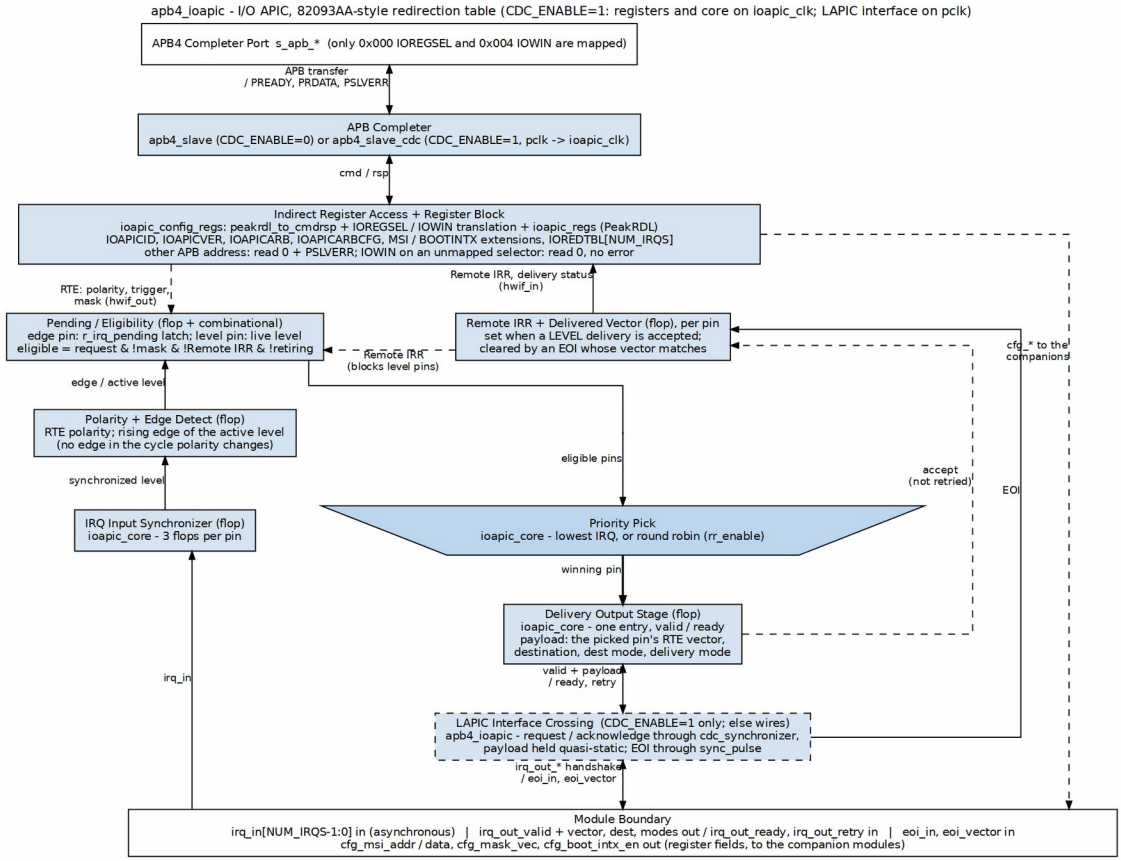
3 ioapic

3.1 Overview

3.1.1 High-Level Architecture

The APB IOAPIC is organized as a hierarchical design with three primary layers:

3.1.2 Figure 1.1: APB IOAPIC Block Diagram



APB IOAPIC Block Diagram

3.2 Functional Description

3.2.1 Block Hierarchy

The design follows RLB architecture standards with clear functional separation:

1. **apb4_ioapic.sv** (Top Level)
    - APB slave interface selection (CDC or non-CDC via parameter)

- Clock domain routing based on CDC_ENABLE
- Module instantiation and wiring
- External interface connections
- LAPIC interface crossing when CDC_ENABLE=1: the delivery handshake and the EOI strobe are presented in pclk and cross into ioapic_clk through matched-latency synchronizers (see Chapter 1.3)

2. **ioapic_config_regs.sv** (Register Interface)
   - PeakRDL adapter instantiation (peakrdl_to_cmdrsp)
   - Generated register block instantiation (ioapic_regs)
   - Special logic for indirect access (IOREGSEL/IOWIN): one selector copy (the regblock field), a decode that admits only APB 0x000 and 0x004, and local acknowledge for every dropped access
   - Hardware interface signal mapping (hwif_in/hwif_out to core)
   - Redirection table array handling (24 entries)
3. **ioapic_core.sv** (Core Logic)
   - IRQ input synchronization (3-stage, metastability prevention)
   - Polarity inversion (active-high/active-low handling; the edge detector is held off for one cycle after a polarity bit changes, so the flip itself is never an edge)
   - Edge detection (rising edge of the polarity-adjusted signal only; there is no falling-edge mode and no filtering)
   - Level sensing (the live synchronized level is the request)
   - Priority arbitration (static: lowest IRQ wins; round robin behind IOAPICARBCFG.rr_enable)
   - One-entry valid/ready delivery stage (no state machine)
   - Remote IRR management (per-pin in-service tracking, EOI matched against the delivered vector)
   - Delivery status per IRQ
4. **ioapic_regs.sv** (Generated by PeakRDL)
   - Direct APB registers: IOREGSEL, IOWIN
   - Internal registers: IOAPICID, IOAPICVER, IOAPICARB
   - Redirection table: IOREDTBL[24] with LO/HI splits
   - Hardware interface structs (hwif_in, hwif_out)

3.2.2 Data Flow

Configuration Path (APB Write): Figure 1.1, down the center:

1. Software writes IOREGSEL, then IOWIN, through `apb4_slave[_cdc]`.
2. `peakrdl_to_cmdrsp` hands the access to the IOREGSEL / IOWIN translation in `ioapic_config_regs`.
3. `ioapic_regs` stores the field; `hwif_out` carries the RTE to `ioapic_core`.

Status Readback Path (APB Read): The same edges carry the response back up:

1. `ioapic_core` reports Remote IRR and delivery status through `hwif_in`.
2. `ioapic_regs` returns the selected register through IOWIN.
3. `peakrdl_to_cmdrsp` and `apb4_slave[_cdc]` return it as PRDATA.

Interrupt Delivery Path: Figure 1.1, up the left edge and down the center:

1. `irq_in` passes the 3-flop synchronizer and the RTE polarity.
2. An edge pin latches its pending bit; a level pin requests while the level is active.
3. Eligible pins (not masked, Remote IRR clear) meet at the priority pick.
4. The winner loads the one-entry output stage, which presents `irq_out_valid` with the vector and destination.
5. An accept (valid, ready, not retried) retires the pin: an edge pin's pending bit clears, a level pin's Remote IRR sets.

EOI Return Path (Level Interrupts): Figure 1.1, up the right edge:

1. `eoi_in` and `eoi_vector` arrive (through `sync_pulse` when CDC_ENABLE=1).
2. Remote IRR clears on EVERY pin whose delivered vector matches.
3. Those pins request again if still asserted; no other pin was ever blocked.

3.2.3 Indirect Access Method

The IOAPIC uses the two-step indirect register access:

Step 1: Select Internal Register

```
Write to IOREGSEL (APB address 0x00):
  Data = Internal register offset (0x00-0x3F)
```

Step 2: Access Selected Register

```
Read/Write IOWIN (APB address 0x04):
  Data = Selected register contents
```

Example: Configure IRQ0 Redirection Entry

```
// Step 1: Select IOREDTBL[0]_LO (internal offset 0x10)
*IOREGSEL = 0x10;

// Step 2: Write configuration via IOWIN
*IOWIN = 0x00000020;  // Vector 0x20, edge-triggered, unmasked

// Step 3: Select IOREDTBL[0]_HI (internal offset 0x11)
*IOREGSEL = 0x11;

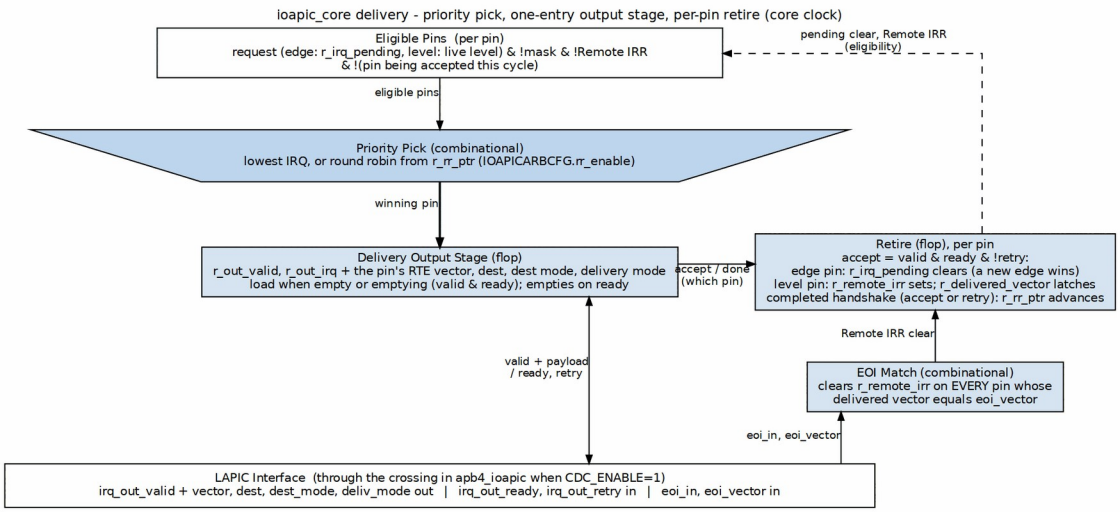
// Step 4: Write destination via IOWIN
*IOWIN = 0x01000000;  // Destination APIC ID = 1
```

This indirect access method: - Reduces address space (only 2 APB registers instead of 50+) - Matches the 82093AA specification for software compatibility - Allows 256 internal registers with 8-bit offset - The IOREGSEL/IOWIN routing is handwritten in ioapic_config_regs.sv (the regblock's IOREGSEL field is the one and only selector, a case remaps APB 0x004, and any address other than 0x000/0x004 is dropped with PSLVERR before the regblock); the PeakRDL block just decodes the translated address. An unmapped selector is stored and readable; IOWIN accesses made while it is unmapped are dropped (read 0, no error)

3.2.4 Interrupt Flow State Machine

There is none. The IOAPIC core delivers through a one-entry valid/ready output stage, and the only per-interrupt state is a Remote IRR bit per pin:

3.2.5 Figure 1.2: IOAPIC Delivery Stage



IOAPIC Delivery Stage

Edge-triggered path: load → present → accept clears pending, exactly once per edge. **Level-triggered path:** load → present → accept sets Remote IRR and frees the stage; the pin stays out of arbitration until the EOI for the vector it was delivered arrives, and re-delivers after that only if the level is still asserted. Nothing about one pin's service state blocks another pin.

The old three-state engine (idle / deliver / wait-for-EOI) was retired with issue #48 (fixed 2026-09-09): its wait state blocked the whole block on one lost EOI, and its delayed pending-clear delivered every edge twice.

3.2.6 Address Space Organization

APB Address Space (12-bit: 0x000-0xFFF): - 0x000: IOREGSEL (software-visible) - 0x004: IOWIN (software-visible) - 0x008-0xFFF: not decoded - and that means the rest of the first 256 bytes as much as 0x100 and above. The access is dropped before the register block and answered locally with PSLVERR (writes ignored, reads return 0). There is no direct path to the internal register file: IOREGSEL/IOWIN is the only way in, and nothing aliases

Internal Register Space (8-bit offset via IOREGSEL): - 0x00: IOAPICID - 0x01: IOAPICVER - 0x02: IOAPICARB - 0x03: IOAPICARBCFG (round-robin enable; not an 82093AA register) - 0x04-0x07: IOAPICMSIADDR, IOAPICMSIDATA, IOAPICMSIDROP, IOAPICBOOTINTX (extensions; reserved selectors on the real 82093AA) - 0x08-0x0F: Reserved (unmapped; IOWIN access dropped) - 0x10-0x3F: IOREDTBL[0-23] (LO/HI pairs)

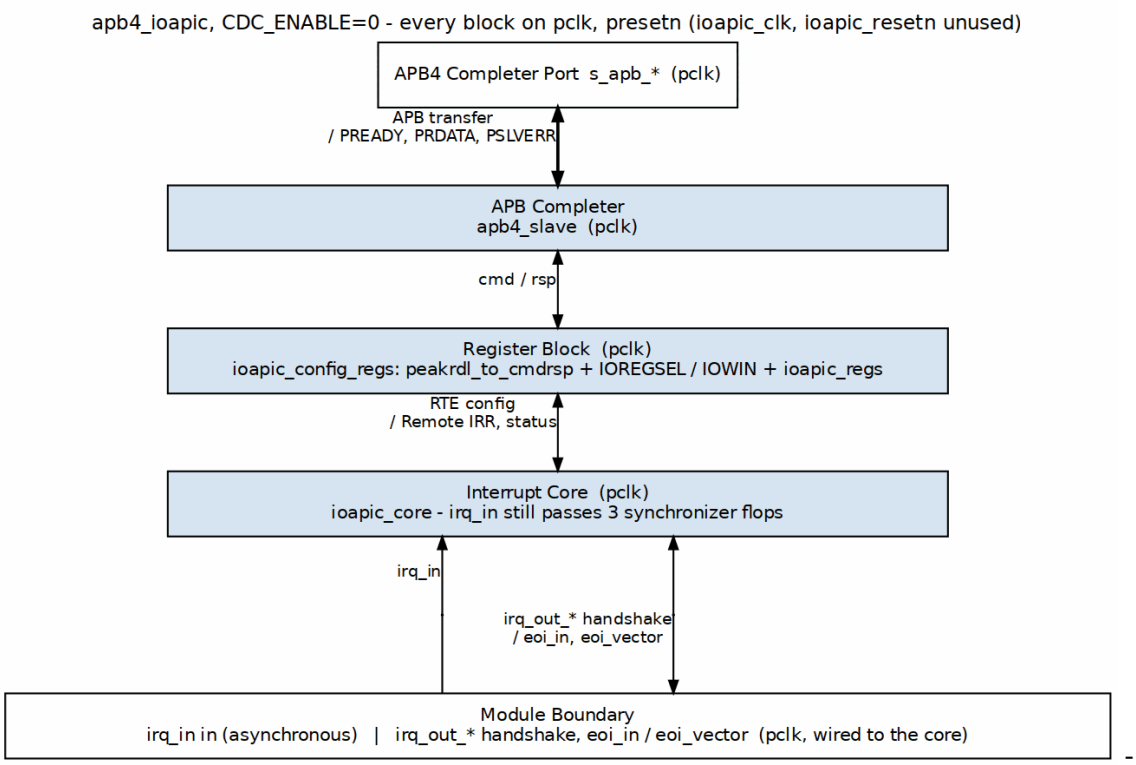
Memory Footprint: 4KB APB window (matches other RLB modules)

3.3 Timing

3.3.1 Clock Domain Architecture

The IOAPIC supports two clock domain configurations via the CDC_ENABLE parameter (a companion USE_JOHNSON parameter, default 0, selects the CDC FIFO pointer encoding and is forwarded to apb4_slave_cdc):

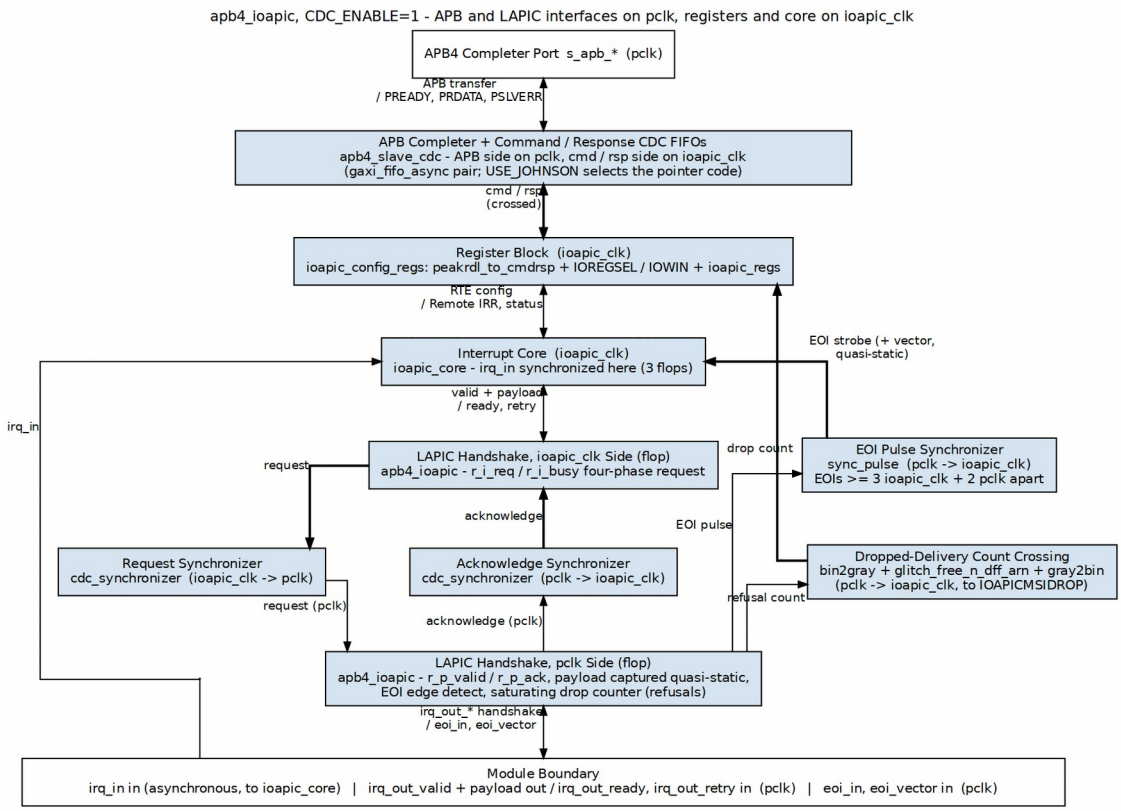
Single Clock Domain (CDC_ENABLE=0 - Default): ### Figure 1.3: Single Clock Domain (CDC_ENABLE=0)



-

Simplest configuration - Lowest latency - Single clock timing analysis - Recommended for most use cases

Dual Clock Domain (CDC_ENABLE=1): ### Figure 1.4: Dual Clock Domain (CDC_ENABLE=1)



- Allows independent APB and IOAPIC clocks - Useful for always-on interrupt capture - ioapic_clk can run while pclk is gated: interrupts are captured (edges latched, levels tracked) and presented once pclk resumes, since the LAPIC-facing interface lives in pclk - Adds 2-4 cycle latency for CDC handshake - The CPU/LAPIC-facing interface (irq_out_valid/ready and payload, eoi_in/eoi_vector) stays in pclk; apb4_ioapic crosses it into ioapic_clk with a four-phase request for deliveries and pulse synchronizers for the accept and the EOI (details in Chapter 1.3)

Clock Selection Logic:

```
// In apb4_ioapic.sv:
.clk ((CDC_ENABLE != 0) ? ioapic_clk : pclk)
```

config_regs and core always share one clock; the only crossings are the APB slave's CMD/RSP FIFOs and the LAPIC interface synchronizers, both in apb4_ioapic.

3.3.2 Reset Architecture

The IOAPIC uses standard RLB reset methodology:

Reset Signals: - `presetn` - APB domain reset (active-low, async) - `ioapic_resetn` - IOAPIC domain reset (active-low, async)

Reset Routing:

```
// In apb4_ioapic.sv:
.rst_n ((CDC_ENABLE != 0) ? ioapic_resetn : presetn)
```

Reset Behavior: - All IRQs masked by default (mask bit = 1) - No interrupts pending - Output stage empty (irq_out_valid = 0) - Remote IRR cleared for all IRQs - IOAPIC ID = 0x0 - All redirection entries reset to safe defaults

3.4 Usage Example

3.4.1 Integration Guidelines

Minimal Integration (Single CPU):

```
apb4_ioapic #(
    .NUM_IRQS    (24),  // must remain 24: the generated decode and
                        // IOAPICVER (MaxRedirEntry=0x17) are
hardwired
    .CDC_ENABLE  (0)   // Single clock domain
) u_ioapic (
    .pclk              (sys_clk),
    .presetn           (sys_resetn),
    .ioapic_clk        (sys_clk),      // Same clock
    .ioapic_resetn     (sys_resetn),   // Same reset

    .s_apb_*           (/* APB signals */),
    .irq_in            (system_irqs),
    .irq_out_valid     (cpu_irq_valid),
    .irq_out_vector    (cpu_irq_vector),
    .irq_out_dest      (/* tie to CPU ID */),
    .irq_out_deliv_mode(/* unused in simple system */),
    .irq_out_ready     (cpu_irq_ack),
    .eoi_in            (cpu_eoi),
    .eoi_vector        (cpu_eoi_vector)
);
```

Advanced Integration (Multi-CPU with CDC):

```
apb4_ioapic #(
    .NUM_IRQS    (24),
    .CDC_ENABLE  (1)   // Dual clock domain
) u_ioapic (
    .pclk              (apb_clk),        // APB bus clock
    .presetn           (apb_resetn),
    .ioapic_clk        (always_on_clk),  // Independent clock
    .ioapic_resetn     (por_resetn),

    .s_apb_*           (/* APB signals */),
    .irq_in            (system_irqs),
    .irq_out_*         (/* to LAPIC router */),
```

```
    .eoi_in             (eoi_from_lapic),
    .eoi_vector         (eoi_vector_from_lapic)
);
```

3.5 Design Notes

3.5.1 Design Trade-offs

Indirect vs Direct Access: - **Chosen**: Indirect access (IOREGSEL/IOWIN) - **Reason**: 82093AA compatibility, reduced address space - **Cost**: Extra APB cycle per access, more complex logic - **Benefit**: Software portability, scalable register space

Static vs Dynamic Priority: - **Chosen**: both, selected by `IOAPICARBCFG.rr_enable`, static at reset - **Reason**: static is the 82093AA scheme and what a driver written for the part expects; round robin is what a system needs when a low-numbered level pin that software EOIs promptly would otherwise starve everything above it - **Cost**: a rotation pointer and a scan that starts at an offset - **Benefit**: the 82093AA behaviour is the default, so nothing changes for software that does not ask for the other one

Fixed vs Multiple Delivery Modes: - **Chosen**: Fixed mode only for MVP - **Reason**: Covers 90%+ of use cases, simpler implementation - **Cost**: Can't use LowestPri, SMI, NMI, etc. yet - **Benefit**: Clean implementation, extensible design

3.6 Navigation

Next: Chapter 1.3 - Clocks and Reset

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

4 ioapic

4.1 Overview

The IOAPIC supports both single and dual clock domain operation via the `CDC_ENABLE` parameter. This chapter covers the clocking options, the reset behavior, and — the part that will bite you if you skip it — exactly which signals are synchronized and which are not.

4.2 Functional Description

4.2.1 Reset Signals

Reset Types:

Signal	Polarity	Type	Domain	Purpose
`presetn`	Active Low	Async	APB	Resets APB interface
`ioapic_resetn`	Active Low	Async	IOAPIC	Resets interrupt logic

Reset Routing (CDC_ENABLE determines which reset is used):

```
// In apb4_ioapic.sv:
assign config_regs_rst = (CDC_ENABLE != 0) ? ioapic_resetn : presetn;
assign core_rst = (CDC_ENABLE != 0) ? ioapic_resetn : presetn;
```

CDC=0: Both use `presetn` **CDC=1:** Both use `ioapic_resetn`

4.2.2 Reset Behavior

On Reset Assertion:

1. **APB Interface:**
   - APB slave returns to IDLE
   - All pending transactions aborted
   - PREADY deasserted
2. **Registers:**
   - IOREGSEL ← 0x00
   - IOAPICID ← 0x00000000
   - All IOREDTBL entries ← default (all IRQs masked)
3. **Core Logic:**
   - All IRQ pending flags cleared

- Output stage empty (irq_out_valid = 0)
- All Remote IRR flags and delivered-vector latches cleared
- Synchronizer chains reset
- No interrupts pending or being delivered

Redirection Table Reset Values: - Vector: 0x00 - Delivery Mode: 0b000 (Fixed) - Dest Mode: 0 (Physical) - Delivery Status: 0 (Idle) - Polarity: 0 (Active High) - Remote IRR: 0 - Trigger Mode: 0 (Edge) - **Mask: 1 (MASKED)** ← Critical: All IRQs disabled by default - Destination: 0x00

After reset, software must: 1. Configure IOAPIC ID (if multiple IOAPICs) 2. Configure each needed IRQ's redirection entry 3. Unmask desired IRQs (clear mask bit)

4.2.3 Reset Sequencing

Power-On Reset:

```
1. Power stabilizes
2. POR circuitry asserts presetn/ioapic_resetn
3. Hold reset for minimum 10 clock cycles
4. Deassert reset synchronously
5. Wait 5 clock cycles for synchronizers
6. Begin software initialization
```

Software Reset (via PM_ACPI if integrated):

```
1. PM_ACPI asserts system reset
2. presetn/ioapic_resetn asserted
3. IOAPIC returns to reset state
4. Software must reinitialize all config
```

Reset the two domains together (CDC_ENABLE=1). Resetting only one side can make a pulse synchronizer in the LAPIC crossing fabricate a single edge. Both consumers qualify what they receive - an accept needs a valid delivery in the stage, an EOI needs a set Remote IRR - so a fabricated pulse after a one-sided reset is a no-op rather than a lost or duplicated interrupt, but the intended sequence is still both resets asserted at once.

4.3 Timing

4.3.1 Clock Domains

4.3.1.1 Single Clock Domain (CDC_ENABLE=0 - Default)

Configuration: - Both APB interface and IOAPIC logic use same clock (`pclk`) - `ioapic_clk` parameter tied to `pclk` - No clock domain crossing logic instantiated

Advantages: - Simplest configuration - Lowest latency (~2 APB cycles for register access) - Single clock timing constraints - Recommended for most applications

Clock Routing: Every block runs on pclk; Figure 1.3 shows the single-clock structure.

4.3.1.2 Dual Clock Domain (CDC_ENABLE=1)

Configuration: - APB interface uses `pclk` - IOAPIC logic uses `ioapic_clk` (independent) - `apb4_slave_cdc` provides clock domain crossing - CMD/RSP signals cross domains via async FIFOs - The CPU/LAPIC-facing interface (`irq_out_valid`, `irq_out_vector`, `irq_out_dest`, `irq_out_deliv_mode`, `irq_out_ready`, `eoi_in`, `eoi_vector`) is presented in `pclk` and crossed inside `apb4_ioapic`

Advantages: - Independent clock frequencies - Can gate `pclk` without losing interrupts: capture (synchronizers, edge latches, level tracking, arbitration) runs in `ioapic_clk`; presentation to the CPU waits for `pclk`, because the LAPIC-facing interface lives there - Useful for power-managed systems

Clock Routing: Figure 1.4 shows the domains and every crossing: the APB command and response FIFOs inside `apb4_slave_cdc`, the LAPIC request and acknowledge synchronizers, the EOI pulse synchronizer and the Gray-coded drop count.

LAPIC Interface Crossing (CDC_ENABLE=1): - Delivery request, ioapic_clk to pclk: classic four-phase handshake. The core's output stage raises a request flop, it crosses a 3-stage synchronizer, and the pclk side registers the payload into the LAPIC-facing `irq_out_*` outputs. The payload is quasi-static behind the handshake - the core cannot change it until the pclk side acks - Accept, pclk to ioapic_clk: the pclk side's ack crosses back through a 3-stage synchronizer and becomes a one-cycle `irq_out_ready` into the core. The request is then withdrawn and the ack withdrawn behind it before the next delivery can start - one full four-phase round trip per delivery, which is what keeps `irq_out_valid` visible for exactly one pclk cycle when the CPU holds `irq_out_ready` high (one handshake, countable) - EOI, pclk to ioapic_clk: `eoi_in` is edge-detected in pclk, the vector is registered on that edge, and the pulse crosses through a 3-stage pulse synchronizer (`sync_pulse`). The vector is quasi-static until the next EOI - Matched latency: the accept and the EOI cross through synchronizers of the same depth, so their pclk ORDER survives into ioapic_clk. That ordering is load-bearing - the core drops an EOI that arrives before the accept it belongs to, and an EOI synchronized against an accept sampled raw in ioapic_clk would have been re-ordered behind it - CDC_ENABLE=0: no crossing; the core drives the LAPIC interface directly and `eoi_in` is consumed as-is (a multi-cycle strobe is harmless there, the Remote IRR clear is idempotent)

Latency Impact: - Register access: +2-4 cycles for CDC handshake - Total: ~4-6 APB cycles vs ~2 cycles for non-CDC - Delivery request to `irq_out_valid`: +1 ioapic_clk (request flop) + 3 pclk (synchronizer) + 1 pclk (LAPIC-facing register) - Back-to-back deliveries: gated by the four-phase round trip (two synchronizer traversals in each direction)

4.3.2 Clock Requirements

Frequency Constraints:

Clock	Minimum	Typical	Maximum	Notes
`pclk`	1 MHz	50-100 MHz	200 MHz	APB bus clock
`ioapic_clk` (CDC=0)	Same as pclk	Same as pclk	Same as pclk	Tied to pclk
`ioapic_clk` (CDC=1)	1 MHz	25-100 MHz	200 MHz	Independent

Relationship (CDC=1): - No fixed relationship required between `pclk` and `ioapic_clk` - Can be asynchronous - Ratio can be arbitrary - CDC logic handles all synchronization

Typical Configurations: - **No CDC:** pclk = ioapic_clk = 100 MHz (system clock) - **With CDC:** pclk = 50 MHz (APB), ioapic_clk = 100 MHz (fast interrupts) - **Power-managed:** pclk = gatable, ioapic_clk = always-on 32 kHz

4.3.3 Interrupt Response Time

From IRQ assertion to delivery request:

Configuration	Synchronization	Edge Detect	Arbitration	Delivery	Total
No CDC, 100 MHz	30 ns (3 cycles)	10 ns (1 cycle)	0 ns (combinational)	10 ns (1 cycle)	~50 ns (5 cycles)
CDC, pclk=50MHz, ioapic_clk=100MHz	30 ns (3 cycles)	10 ns (1 cycle)	0 ns (combinational)	10 ns (1 cycle) + crossing: 10 ns (1 ioapic_clk) + 80 ns (4 pclk)	~140 ns

Note: Above is from IRQ pin to `irq_out_valid`. CPU interrupt latency depends on LAPIC design.

4.3.4 Timing Constraints

Critical Paths (for synthesis):

Without CDC: - APB write → register update → core config: Single clock domain - IRQ input → synchronizer → edge detect → arbitration → delivery: ~4-5 levels of logic - Typical Fmax: 150-200 MHz (depends on synthesis settings)

With CDC: - APB domain: Same as without CDC - IOAPIC domain: IRQ path same as above - Cross-domain: APB CMD/RSP through async FIFOs; LAPIC interface through the four-phase request and pulse synchronizers (no combinational paths) - Constrain the quasi-static payloads: with

CDC_ENABLE=1 the delivery payload buses (vector, dest, delivery mode - ioapic_clk into pclk) and the registered EOI vector (pclk into ioapic_clk) are multi-cycle-path crossings held stable behind the four-phase and pulse handshakes. A build must declare them as such - `set_max_delay -datapath_only` across the crossing, or `set_false_path` - or they show up as timing failures against a clock they were never meant to meet. rlb_top uses CDC_ENABLE=0 and has none of these paths - Typical Fmax: 150-200 MHz each domain independently

Setup/Hold: - IRQ inputs: Asynchronous, 3-stage internal sync - LAPIC interface (`irq_out_*`, `irq_out_ready`, `eoi_in`, `eoi_vector`): synchronous to `pclk` in both configurations. With CDC_ENABLE=1 the crossing into ioapic_clk is inside the block (fixed 2026-09-09, issue #48 - the EOI used to reach the core with no synchronizer at all) - EOI rate contract (CDC_ENABLE=1): consecutive `eoi_in` strobes at least 3 x T_ioapic_clk + 2 x T_pclk apart (the pulse synchronizer's spacing requirement). Closer than that, the two strobes land inside one destination sample window, toggle the pulse synchronizer twice, and BOTH are lost silently - not one - leaving two level pins in service with nothing reported. An EOI must also be at least one pclk cycle away from the delivery accept it is ordered against. One EOI per delivered interrupt and a delivery round trip costs far more than that, so software cannot legitimately hit either limit - APB signals: Per APB specification

4.3.5 Clock Jitter and Stability

IRQ Input Synchronization: - 3-stage synchronizer handles moderate jitter - MTBF (Mean Time Between Failures): >1000 years at 100 MHz - No special jitter requirements on IRQ inputs

Clock Source Requirements: - Stable clock (< 100 ppm drift typical) - Low jitter for timing-critical applications - FPGA PLLs/MMCMs acceptable

4.4 Design Notes

4.4.1 Clock Gating Considerations

With CDC_ENABLE=1:

Can gate pclk when: - No APB accesses needed - Power saving mode - IOAPIC capture still operational on `ioapic_clk` - Interrupts are held, not lost: an edge stays latched, a level stays tracked, and a delivery already in the output stage waits in the four-phase request until `pclk` resumes. Nothing is presented to the CPU, and no EOI is consumed, while `pclk` is stopped - so a wake-up source must be routed outside this interface

Cannot gate ioapic_clk when: - Interrupts must be serviced - Need real-time interrupt response - Unless entering deep power-down (then reinit required)

Integration with PM_ACPI: If using PM_ACPI power management: - Connect IOAPIC to always-on power domain - Use `ioapic_clk` from always-on clock tree - Enable CDC (CDC_ENABLE=1) - IOAPIC continues operating in S1/S3 sleep states

4.5 Navigation

See Also: - Architecture - Clock domain architecture diagrams

Next: Chapter 1.4 - Acronyms

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 ioapic

5.1 Overview

The alphabet soup, decoded. If you've worked with legacy interrupt controllers before, most of this is familiar — the IOAPIC-specific terms are the ones worth a skim.

5.2 References

5.2.1 Acronyms

Acronym	Full Name	Description
IOAPIC	I/O Advanced Programmable Interrupt Controller	advanced interrupt routing controller
APB	Advanced Peripheral Bus	AMBA low-power peripheral bus
AMBA	Advanced Microcontroller Bus Architecture	ARM bus specification
APIC	Advanced Programmable Interrupt Controller	Generic term for advanced interrupt controllers
LAPIC	Local Advanced Programmable Interrupt Controller	Per-CPU interrupt controller
IRQ	Interrupt Request	Hardware interrupt signal
EOI	End of Interrupt	Signal that interrupt service is complete
PIC	Programmable Interrupt Controller	Legacy 8259-style interrupt controller
ISA	Industry Standard Architecture	Legacy PC bus
PCI	Peripheral Component Interconnect	Standard expansion bus
RLB	Retro Legacy Blocks	Project name for classic

Acronym	Full Name	Description
		peripheral IP cores
CDC	Clock Domain Crossing	Synchronization between clock domains
IRR	Interrupt Request Register	standard term for interrupt pending state
ISR	Interrupt Service Routine	Software interrupt handler
SMI	System Management Interrupt	Special interrupt for system management mode
NMI	Non-Maskable Interrupt	High-priority interrupt that can’t be masked
INIT	Initialization	Processor initialization sequence
ExtINT	External Interrupt	8259-compatible interrupt delivery

5.2.2 Register Names

Name	Full Name	Description
IOREGSEL	I/O Register Select	Indirect access register selector
IOWIN	I/O Window	Indirect access data window
IOAPICID	I/O APIC Identification	IOAPIC ID register
IOAPICVER	I/O APIC Version	Version and capabilities register
IOAPICARB	I/O APIC Arbitration	Bus arbitration priority register
IOREDTBL	I/O Redirection Table	Interrupt redirection configuration

5.2.3 Key Terms

Redirection Table: Array of 64-bit entries (one per IRQ) that configure how each interrupt is routed to CPUs. Contains vector, delivery mode, destination, trigger type, polarity, and mask.

Indirect Access: The register access method where software writes an offset to IOREGSEL, then accesses the selected register via IOWIN. Reduces address space requirements.

Remote IRR (Interrupt Request Register): Status bit for level-triggered interrupts. Set when interrupt is accepted by CPU, cleared when EOI is received. Prevents interrupt re-triggering while being serviced.

Delivery Status: Read-only bit indicating if an interrupt delivery is in progress for a specific IRQ. Used to monitor interrupt flow.

Trigger Mode: - **Edge:** Interrupt on signal edge (rising after polarity adjustment) - **Level:** Interrupt on signal level (requires EOI to clear)

Polarity: - **Active High:** Interrupt when signal is logic 1 - **Active Low:** Interrupt when signal is logic 0 (inverted internally)

Delivery Mode: Determines how interrupt is delivered to CPU: - **Fixed:** To specific CPU - **Lowest Priority:** To the least-busy CPU in the destination set. The IOAPIC does not choose; the local APICs arbitrate on their Arbitration Priority Registers and one accepts, and `irq_out_retry` reports the case where none could - **SMI/NMI/INIT/ExtINT:** Special modes

Destination Mode: - **Physical:** Destination field is physical APIC ID - **Logical:** Destination field is logical grouping (multi-cast)

Priority Arbitration: When multiple IRQs are pending, hardware selects which to deliver first. Two policies: static priority (lowest IRQ number wins), which is the reset default, or round robin from the last accepted pin when `IOAPICARBCFG.rr_enable` is set.

End-of-Interrupt (EOI): Signal from CPU indicating interrupt service is complete. For level-triggered interrupts, clears Remote IRR on every pin delivered with that vector and allows re-triggering if signal still asserted.

5.3 Navigation

See Also: - References - External standards and specifications - Overview - Component introduction - Architecture - System architecture

6 ioapic

6.1 Overview

Everything this spec leans on: the standards, the sibling RLB documents, the tools, and — for the historically curious — how PC interrupt controllers got here.

6.2 References

6.2.1 Primary Standards

1. **82093AA I/O Advanced Programmable Interrupt Controller (IOAPIC)**
   - (publisher)
   - May 1996
   - Document Order Number: 290566-001
   - **Purpose:** Defines IOAPIC register layout, indirect access method, and behavior
   - **Relevance:** APB IOAPIC implements this specification with APB interface
2. **AMBA APB Protocol Specification**
   - ARM Limited
   - AMBA 4 APB Protocol Specification v2.0
   - **Purpose:** Defines APB4 bus protocol
   - **Relevance:** APB interface implementation
3. **SystemRDL 2.0 Specification**
   - Accellera Systems Initiative
   - **Purpose:** Register description language
   - **Relevance:** PeakRDL register generation from .rdl specifications

6.2.2 Related RLB Documentation

Module-Specific: - `../../rtl/ioapic/README.md` - Block summary; the feature arc is RLB TASK-008 (was RLB-008) in `vault/Tasks/RLB/closed.md` - `../../rtl/ioapic/README.md` - PeakRDL register generation guide - `../../rdl/ioapic/ioapic_regs.rdl` - SystemRDL source specification

RLB System: - `../../../../../../vault/Tasks/RLB/RLB_MODULE_AUDIT.md` - Architecture compliance verification - `../../../../../../vault/Tasks/RLB/RLB_STATUS_AND_ROADMAP.md` - System-wide status

and planning - `../../RLB_FPGA_IMPLEMENTATION_GUIDE.md` - FPGA deployment guide - `../../PRD.md` - Product Requirements Document - `../../CLAUDE.md` - AI integration guide

Reference RLB Modules: - `../../hpet_mas/hpet_mas_index.md` - HPET specification (architectural reference) - `../../rtl/pic_8259/README.md` - Legacy PIC implementation - `../../rtl/pm_acpi/README.md` - Power management and ACPI

6.2.3 External Resources

APIC Architecture: - MultiProcessor Specification Version 1.4 - Architecture Software Developer's Manual (Volume 3: System Programming) - **Relevance:** APIC system architecture, LAPIC integration, interrupt delivery

AMBA Specifications: - AMBA APB Protocol Specification v2.0 (ARM IHI 0024) - AMBA AXI and ACE Protocol Specification (for AXI-APB bridges) - **Relevance:** Bus protocol compliance

SystemRDL Tools: - PeakRDL-regblock Documentation - PeakRDL-html Documentation - SystemRDL 2.0 Language Reference Manual - **Relevance:** Register generator usage

6.2.4 Design Methodology References

RLB Architecture Pattern:

```
APB → apb4_slave[_cdc] → CMD/RSP → peakrdl_to_cmdrsp →
  → <module>_regs (PeakRDL) → hwif → <module>_core
```

Reference Implementations: - HPET: Complete reference with CDC, PeakRDL, documentation - PM_ACPI: Recent implementation, similar complexity - SMBus: Protocol controller with FIFOs

Converter Components: - `../../converters/rtl/apb4_slave.sv` - APB slave without CDC - `../../converters/rtl/apb4_slave_cdc.sv` - APB slave with CDC - `../../converters/rtl/peakrdl_to_cmdrsp.sv` - PeakRDL adapter

6.2.5 Historical Context

Evolution of PC Interrupt Controllers:

1. **8259A PIC (1976-1990s)**
   - 8 IRQ inputs, cascadable to 15
   - Fixed priority
   - Edge-triggered by default (ICW1 LTIM selects a level-triggered mode)
   - Direct memory-mapped access
2. **82093AA IOAPIC (1996-present)**
   - 24 IRQ inputs
   - Programmable priority
   - Edge and level-triggered

- Indirect register access
- Multi-processor support
- **This implementation**

3. **MSI/MSI-X (2000s-present)**
   - Message-based interrupts
   - No dedicated IRQ lines
   - Scalable to thousands of interrupts
   - PCIe standard

APB IOAPIC Position: Modern implementation of 82093AA specification using RLB methodology, suitable for soft-core SoCs, FPGA systems, and PC-compatible designs.

6.2.6 Software Examples and Drivers

Linux Kernel: - `arch/x86/kernel/apic/io_apic.c` - Linux IOAPIC driver - Shows real-world usage patterns - Demonstrates edge/level handling, EOI, redirection table setup

xv6 Operating System: - `ioapic.c` - Simple IOAPIC driver for educational OS - Clean example of indirect access - Good reference for basic initialization

SeaBIOS / **Coreboot:** - BIOS-level IOAPIC initialization - Shows boot-time configuration - Demonstrates hardware discovery

6.2.7 Verification References

Cocotb (Python-based Verification): - Cocotb documentation: https://docs.cocotb.org/ - Used for RLB module validation - Tests in `../../dv/tests/` directories

Verification Strategy: - Similar to HPET: Basic → Medium → Full test hierarchy - Test levels defined in RLB_STATUS_AND_ROADMAP.md - Estimated 5-7 days for complete validation

6.3 Navigation

Document Navigation: - Overview - Architecture - Acronyms

7 ioapic

7.1 Overview

7.1.1 Module Hierarchy

The APB IOAPIC consists of four primary blocks organized in a clean hierarchical structure:

```
apb4_ioapic (Top Level)
├── apb4_slave or apb4_slave_cdc (Bus Interface)
├── ioapic_config_regs (Register Wrapper)
│   ├── peakrdl_to_cmdrsp (Protocol Adapter)
│   └── ioapic_regs (PeakRDL Generated)
└── ioapic_core (Core Interrupt Logic)
```

7.1.2 Block Summary

Block	File	Lines	Purpose
apb4_ioapic	apb4_ioapic.sv	~560	Top-level integration, CDC selection, LAPIC interface crossing
ioapic_config_regs	ioapic_config_regs.sv	~410	Register interface, indirect access, window decode, hwif mapping
ioapic_regs	ioapic_regs.sv	~500	PeakRDL generated register block
ioapic_core	ioapic_core.sv	~460	Interrupt routing, edge/level detection, arbitration, delivery stage
peakrdl_to_cmdrsp	(external)	~150	CMD/RSP to PeakRDL passthrough adapter
apb4_slave[_cdc]	(external)	~200	APB protocol handler

Total Implementation: ~1400 lines of custom RTL + ~1000 lines generated/reused

7.1.3 Block Descriptions

1. apb4_ioapic (Top Level) - Selects APB slave type based on CDC_ENABLE parameter - Routes clocks and resets to submodules - Instantiates config_regs and core - Connects external IRQ and EOI interfaces - CDC_ENABLE=1: presents the whole LAPIC-facing interface in pclk and crosses it into ioapic_clk (four-phase delivery request, 3-stage accept synchronizer, 3-stage EOI pulse synchronizer with the vector registered)

2. ioapic_config_regs (Register Wrapper) - Instantiates peakrdl_to_cmdrsp adapter - Instantiates PeakRDL generated registers - Translates IOWIN accesses through the one selector copy (the regblock's IOREGSEL field); drops IOWIN accesses with an unmapped selector (read 0, no error) and any access to an address other than 0x000/0x004 (PSLVERR), acknowledging both locally - Maps hwif signals to/from ioapic_core - Handles array mapping for 24 redirection entries

3. ioapic_regs (PeakRDL Generated) - Generated from ioapic_regs.rdl SystemRDL specification - Provides hwif_in/hwif_out structs (the IOREGSEL/IOWIN indirect-access translation is HANDWRITTEN in ioapic_config_regs.sv – the generated block only holds standalone IOREGSEL/IOWIN registers; its IOWIN storage is never reached through the bus, since 0x004 accesses are always translated away, and its hardware input is tied to zero so the field is defined rather than X) - Handles register read/write/reset

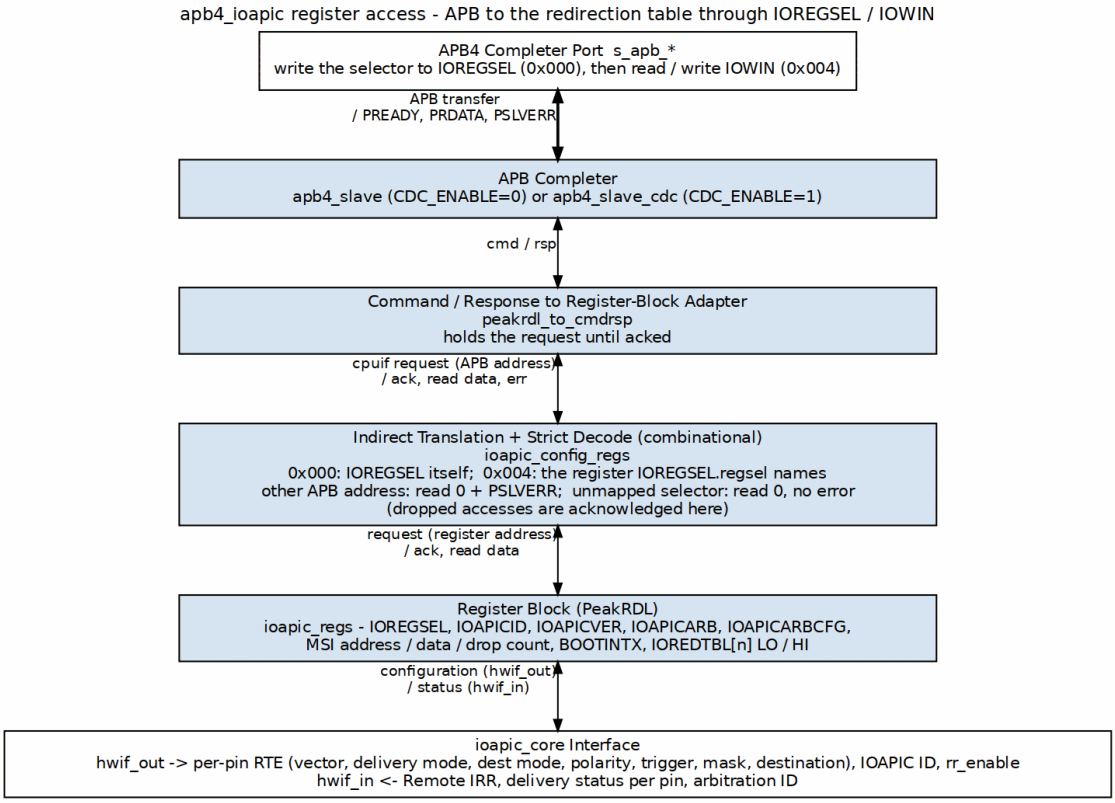
4. ioapic_core (Core Logic) - 24 IRQ input synchronization (3-stage) - Polarity handling (active-high/low) - Edge/level detection - Priority arbitration (static) - One-entry valid/ready delivery stage (no state machine) - Remote IRR management, per pin, EOI matched against the delivered vector

7.2 Functional Description

7.2.1 Data Flow Between Blocks

Configuration Write Path:

7.2.2 Figure 2.1: IOAPIC Register Access Path



IOAPIC Register Access Path

Status Read Path:

Status readback rides the same edges of Figure 2.1 upward: `ioapic_core` status into `hwif_in`, the register block returns the selected register through IOWIN, and the adapter and APB completer return it as PRDATA.

Interrupt Delivery Path:

The interrupt path is Figure 1.1: `irq_in` climbs through the synchronizer and edge / level request, the priority pick loads the output stage, and the delivery leaves through the LAPIC interface (crossed into pclk when CDC_ENABLE=1). The accept and the EOI come back the same way.

7.2.3 Interface Summary

External Interfaces: - APB4 slave (to CPU/interconnect) - 24 IRQ inputs (from interrupt sources, asynchronous) - Interrupt output, valid/ready (to CPU/LAPIC, pclk domain) - EOI input (from CPU/LAPIC, pclk domain)

Internal Interfaces: - CMD/RSP between APB slave and config_regs - PeakRDL passthrough between adapter and registers - hwif between registers and config_regs - Config/status signals between config_regs and core

7.3 Timing

7.3.1 Clock Domain Assignment

CDC_ENABLE=0 (Single Clock): - All blocks run on `pclk` - apb4_slave instantiated (no CDC) - Simplest timing analysis

CDC_ENABLE=1 (Dual Clock): - apb4_slave_cdc runs on `pclk` (APB domain) - config_regs runs on `ioapic_clk` (IOAPIC domain) - core runs on `ioapic_clk` (IOAPIC domain) - APB CDC handled by apb4_slave_cdc module - LAPIC interface CDC handled in apb4_ioapic (delivery request, accept and EOI cross through matched-latency synchronizers; the external ports stay in `pclk`)

7.4 Related Modules

7.4.1 Module Dependencies

RLB Project Dependencies: - `reset_defs.svh` - Reset macro definitions - `apb4_slave.sv` or `apb4_slave_cdc.sv` - APB protocol - `peakrdl_to_cmdrsp.sv` - Register adapter

Generated Files: - `ioapic_regs.sv` - From ioapic_regs.rdl via peakrdl_generate.py - `ioapic_regs_pkg.sv` - Package with hwif struct definitions

No External IP Required: All dependencies are within the RLB project or generated from specifications.

7.5 Navigation

Chapter 2 Contents: - 05_fsm_summary.md - State machine summary

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 ioapic

8.1 Overview

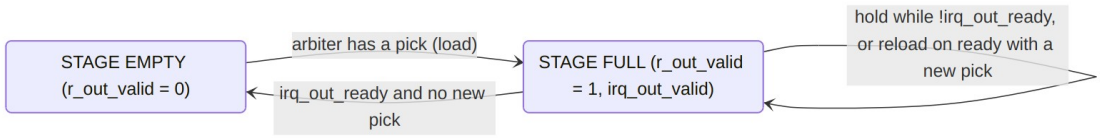
The IOAPIC core has no delivery state machine. The delivery path is a one-entry valid/ready output stage fed by the arbiter, and the only other state that matters is a Remote IRR bit per pin. That is a smaller thing than the three-state FSM this chapter used to describe (idle / deliver / wait-for-EOI, retired with issue #48 on 2026-09-09), and it is smaller for a reason: the three states carried nothing the handshake and the per-pin bits do not. The corner cases still live in the edge/level split and the Remote IRR bookkeeping, so this is still the page to have open while reading `ioapic_core.sv`.

8.2 Functional Description

8.2.1 State Definitions

State	Where it lives	Description
Stage empty	`r_out_valid = 0`	Nothing presented to the CPU; the arbiter's pick, if any, loads next cycle
Stage full	`r_out_valid = 1`	One delivery presented on `irq_out_valid`/vector/dest; waiting for `irq_out_ready`
In service	`r_remote_irr[i] = 1`	Level pin `i` has been accepted and not yet EOI'd; that pin alone is masked from arbitration

The old encoding mapped onto these as: idle = stage empty, deliver = stage full, and wait-for-EOI = "some pin in service". The difference is that the old wait state blocked everything, and "in service" blocks one pin.



State Transition Diagram

On the accept (`r_out_valid && irq_out_ready && !irq_out_retry`) the retiring pin i, in that same cycle:

- edge pin: `r_irq_pending[i] <= 0` (a new edge wins)
- level pin: `r_remote_irr[i] <= 1` (pin in service)
- both: `r_delivered_vector[i] <= r_out_vector`
- pin i is masked from this cycle's arbitration

Per level pin, independent of the stage: `r_remote_irr[i]` goes 0 to 1 on the accept and back to 0 on `eoi_in && eoi_vector == r_delivered_vector[i]`.

8.2.2 State Transitions

Current State	Condition	Next State	Action
Stage empty	No eligible pin	Stage empty	Keep arbitrating
Stage empty	Eligible pin found	Stage full	Register pick: index, vector, dest, delivery mode, trigger mode
Stage full	!irq_out_ready	Stage full	Hold; outputs are registered and do not change
Stage full	irq_out_ready, another pin eligible	Stage full	Accept the current one, load the next in the same cycle (no bubble)
Stage full	irq_out_ready, nothing eligible	Stage empty	Accept, drop `irq_out_valid`
Remote IRR[i] = 0	Accept of level pin i	Remote IRR[i] = 1	Pin i leaves arbitration; the stage is free immediately
Remote IRR[i] = 1	eoi_in && eoi_vector == delivered vector of pin i	Remote IRR[i] = 0	Pin i re-enters arbitration next cycle if still asserted
Remote IRR[i] = 1	EOI with any other vector	Remote IRR[i] = 1	Other pins unaffected; only pin i stays blocked

8.2.3 State Functions

Stage empty: - **Entry:** From stage full when the CPU accepts and no other pin is eligible; from reset - **Operations:** - Scan all 24 pins for eligible requests (requesting, unmasked, not in service, not being retired this cycle) - Apply priority arbitration (lowest IRQ number wins) - Load the winner into the output stage - **Outputs:** - irq_out_valid = 0; vector, dest and delivery mode read as 0 - **Exit:** When an eligible pin exists

Stage full: - **Entry:** From stage empty on a load, or from stage full when an accept and a load coincide - **Operations:** - Hold irq_out_valid with the registered vector, destination and delivery mode - Wait for the CPU (irq_out_ready) - **Outputs:** - irq_out_valid = 1 - irq_out_vector = the vector registered at load time - irq_out_dest = the destination registered at load time - irq_out_deliv_mode = the delivery mode registered at load time - (registered, so a mid-delivery RTE rewrite does NOT change what is presented; the next delivery of that pin uses the new values) - **Exit:** On accept (`r_out_valid && irq_out_ready`), every time - one accept retires exactly one delivery. Edge pins clear their pending latch on that accept, in the same cycle, which is what makes each edge deliver once.

In service (level pins, per pin): - **Entry:** Accept of a level delivery - not presentation. An EOI that arrives while the delivery is still unaccepted finds Remote IRR clear and is dropped rather than pre-clearing anything - **Operations:** - Remote IRR set; the pin is masked from arbitration - The input is still synchronized, so its live level is visible the moment Remote IRR clears - Every other pin keeps arbitrating and delivering - **Outputs:** - status_remote_irr[i] = 1 - **Exit:** EOI whose vector equals the vector delivered on this pin (latched at accept). The compare runs on every pin, so one EOI clears every pin delivered with that vector - two pins sharing a vector leave service together. A lost or wrong-vector EOI leaves this pin, and only this pin, blocked.

8.2.4 Latched Signals

Signals registered when the stage loads:

Signal	Source	Purpose
r_out_irq[4:0]	w_sel_irq	Pin being delivered; selects which pending/Remote IRR bit the accept touches
r_out_vector[7:0]	cfg_vector[w_sel_irq]	Presented on irq_out_vector; copied into r_delivered_vector on accept
r_out_dest[7:0]	cfg_destination[w_sel_irq]	Presented on irq_out_dest
r_out_deliv_mode[2:0]	cfg_deliv_mode[w_sel_ir	Presented on

Signal	Source	Purpose
	q]	irq_out_deliv_mode

Signal registered on accept, per pin:

Signal	Source	Purpose
r_delivered_vector[i][7:0]	r_out_vector	The vector the CPU actually received; the EOI is matched against this, not the live RTE

These remain stable while the stage is full; the delivered vector stays until the pin's next accept.

8.2.5 Edge vs Level Interrupt Paths

Edge-Triggered Interrupt: 1. The IRQ asserts; the rising edge of the polarity-adjusted level sets `r_irq_pending`. 2. Arbitration selects it, the stage loads, and `irq_out_valid` rises. 3. The CPU accepts; pending clears in the same cycle (a coincident new edge sets it instead). Done. **Time:** 3 sync cycles + 1 edge-detect cycle + 1 load cycle to `irq_out_valid`; the delivery retires on the first cycle the CPU holds `irq_out_ready`.

Polarity: flipping an RTE's polarity bit inverts the active level, which would read as a rising edge - so the edge detector for that pin is suppressed for one cycle after the bit changes, and the flip itself never latches a pending interrupt. Software may change polarity on an idle pin without a spurious delivery.

Level-Triggered Interrupt: 1. The IRQ asserts; the synchronized level is the request (no latch, gated by Remote IRR). 2. Arbitration selects it, the stage loads, and `irq_out_valid` rises. 3. The CPU accepts: Remote IRR sets, the delivered vector is latched, and the stage is free. Other pins deliver meanwhile. 4. An EOI with the delivered vector clears Remote IRR. 5. If the level is still asserted, it re-requests the next cycle and is delivered again, once. **Time:** Same as edge to the accept; the pin then waits for the ISR and its EOI, but the block does not.

8.2.6 Arbitration Logic

Priority Encoding (Static Priority):

```
// In ioapic_core.sv
for (int j = 0; j < NUM_IRQS; j++) begin
    if (w_irq_eligible[j]) begin
        w_sel_irq   = IRQ_IDX_W'(j);
        w_sel_valid = 1'b1;
        break;  // Stop at first match (lowest number)
    end
end
```

Eligibility Criteria:

```
// Edge pins request from the latch, level pins from the live
synchronized level
assign w_irq_request[i]  = cfg_trigger_mode[i] ? w_irq_active[i] :
r_irq_pending[i];

assign w_irq_eligible[i] = w_irq_request[i]
                         && !cfg_mask[i]
                         && !r_remote_irr[i]
                         && !(w_deliv_accept && (r_out_irq ==
IRQ_IDX_W'(i)));
```

The last term masks the pin being accepted from the arbitration happening in the same cycle. Without it the retiring pin is still requesting while its latch is being cleared (or its Remote IRR set) and wins one more round - exactly the double delivery of issue #48.

Arbitration Timing: - Combinational (< 1 clock cycle) - Result loads into the output stage on the next edge whenever the stage is empty or being accepted:

```
assign w_out_load = w_sel_valid && (!r_out_valid || irq_out_ready);
```

8.2.7 Remote IRR Management

Set Conditions:

```
// For level-triggered IRQs only, on the ACCEPT of this pin's
delivery.
// The LIVE trigger mode arms this branch, not a copy sampled at load:
// an edge->level rewrite during an in-flight delivery must still land
// in service, or the live level would re-deliver forever.
if (cfg_trigger_mode[i] == 1'b1) begin
    if (w_deliv_accept && (r_out_irq == IRQ_IDX_W'(i))) begin
        r_remote_irr[i] <= 1'b1;
    end
```

Clear Conditions:

```
    end else if (eoi_in && (eoi_vector == r_delivered_vector[i]))
begin
        r_remote_irr[i] <= 1'b0;   // matched against the DELIVERED
vector, not cfg_vector
    end
end
```

An accept and a matching EOI in the same cycle: the accept wins, because the pin has just re-entered service. An EOI for a pin whose Remote IRR is already clear is a no-op. The compare is per pin and every pin runs it on the same EOI, so two level pins delivered with the same vector both clear on one EOI, as on the 82093AA - sharing a vector is legal, it just means one EOI retires both.

Effect on Eligibility:

```
// Level mode: the live level requests, but only while this pin is not
in service
w_irq_eligible[i] = w_irq_active[i] && !cfg_mask[i] && !
r_remote_irr[i] && ...;
```

This prevents a level interrupt from re-triggering while it is being serviced, and stops there: nothing about pin i's service state touches pin j.

8.2.8 Multiple Pending IRQs

Scenario: Multiple IRQs asserted simultaneously

Behavior: 1. Arbitration selects the lowest numbered eligible pin 2. The stage presents it (`irq_out_valid`) 3. Other pins remain requesting 4. On accept, the retiring pin drops out (latch cleared or Remote IRR set) and the next lowest loads in the same cycle 5. Level pins in service are skipped; their EOIs arrive whenever the ISRs finish and do not gate anyone else 6. Process repeats until nothing is eligible

Example Timeline:

Cycle	Event
0	IRQ3 (level), IRQ5 (edge) and IRQ7 (level) all become eligible
1	The stage loads IRQ3
2	The CPU accepts IRQ3: Remote IRR[3] sets; the stage loads IRQ5 in the same cycle
3	The CPU accepts IRQ5: pending[5] clears; the stage loads IRQ7
4	The CPU accepts IRQ7: Remote IRR[7] sets; the stage is empty
…	
N	EOI (vector of IRQ7): Remote IRR[7] clears; IRQ3 is still in service, unaffected
N+1	IRQ7 re-requests if its level is still asserted

Fairness: with static priority, lower numbered IRQs starve higher ones if they keep requesting, and a low-numbered level pin that is EOI'd promptly can hold the stage indefinitely. That is the 82093AA scheme and it is the reset default.

Setting `IOAPICARBCFG.rr_enable` starts the scan just above the pin that was last ACCEPTED, and wraps, so the pin just served is the last one the scan reaches and every eligible pin is served before any pin is served twice. Priority becomes a position in the rotation rather than an IRQ number. The pointer moves on a completed handshake, accepted or retried, and never on a stall: a pick the consumer never takes must not move it, or a stalled consumer would walk the rotation round the ring without delivering anything, while moving on a retry stops a refused pin from monopolising the channel.

8.3 Navigation

Back to: Block Overview

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

9 ioapic

9.1 Overview

9.1.1 Register Access Method

The IOAPIC uses **indirect register access** following 82093AA specification:

1. **Write to IOREGSEL** (APB address 0x00): Select internal register offset
2. **Access IOWIN** (APB address 0x04): Read or write selected register data

Two transactions for every internal register access. Yes, it's clunky. Yes, it's what legacy PC software expects, and that's the point.

9.2 Functional Description

9.2.1 Direct APB Registers

APB Address	Register	Type	Reset	Description
0x000	IOREGSEL	RW	0x00	Register offset selector (0x00-0x3F)
0x004	IOWIN	RW	0x00000000	Data window for selected internal register
0x008-0x0FF	Not decoded	-	-	Dropped before the register block and acknowledged locally with PSLVERR; reads return 0, writes touch nothing. There is no direct path to the internal

APB Address	Register	Type	Reset	Description
				register file
0x100-0xFFF	Not decoded	-	-	Same treatment: dropped with PSLVERR, reads return 0. (Fixed 2026-09-09, issue #48: these used to alias onto 0x000-0x0FF every 256 bytes)

No direct path to the register block. IOREGSEL and IOWIN are the only two software-visible addresses in the 4 KB window - 82093AA semantics. Every other address, 0x008 upward, inside the first 256 bytes as much as above them, is dropped in `ioapic_config_regs.sv` before the register block ever sees it: the write is ignored, the read returns 0, and the access is answered with PSLVERR. IOAPICID, IOAPICVER, IOAPICARB and the redirection table are reachable only through the IOREGSEL/IOWIN pair. The one dropped access that does not raise PSLVERR is an IOWIN access with an unmapped selector, described next.

9.2.2 IOREGSEL Register (APB 0x000)

Bits	Name	Type	Reset	Description
[7:0]	regsel	RW	0x00	Internal register offset for indirect access
[31:8]	Reserved	RO	0x000000	Reserved, read as 0

Valid offset values: - 0x00: IOAPICID - 0x01: IOAPICVER - 0x02: IOAPICARB - 0x03: IOAPICARBCFG (NOT an 82093AA register – see below) - 0x04: IOAPICMSIADDR (NOT an 82093AA register – see below) - 0x05: IOAPICMSIDATA (NOT an 82093AA register – see below) - 0x06: IOAPICMSIDROP (NOT an 82093AA register – see below) - 0x07: IOAPICBOOTINTX (NOT an 82093AA register – see below) - 0x10-0x3F: IOREDTBL entries (even=LO, odd=HI)

Unmapped selector values are stored, readable, and inert. A selector in 0x08-0x0F or at or above 0x40 is accepted by IOREGSEL and reads back as written (there is exactly one copy of the selector - the register block's `regsel` field drives both readback and the IOWIN translation, and

byte strobes are honoured by the register block). An IOWIN access made while the selector is unmapped is dropped: reads return 0, writes touch nothing, no PSLVERR (the address is legal; the selector merely names a register the 82093AA does not implement). The selector itself is unaffected by the dropped access. (Fixed 2026-09-09, issue #48: a second, functional copy of the selector used to diverge from the readback copy on this path.)

Changing polarity on an idle pin is safe; mask before changing trigger mode. Flipping INTPOL inverts the internal active level, which would read as a rising edge - so the edge detector for that pin is suppressed for one cycle after the polarity bit changes, and the flip itself never latches a pending interrupt. Switching trigger mode on a live input has no such guard: an asserted input that becomes level-triggered is delivered at once, which is correct but may not be what you intended. Set the mask bit, reprogram, then unmask (same sequence the real 82093AA asks for).

A lost or wrong-vector EOI blocks only its own pin. Accepting a level interrupt sets that pin's Remote IRR and frees the delivery stage at once; the pin stays out of arbitration until an EOI carrying the vector it was delivered arrives, and every other pin keeps delivering meanwhile. If software EOIs the wrong vector or never EOIs, that one input is blocked until the right EOI arrives, or until it is reprogrammed through a reset - the same behaviour as the 82093AA. (Fixed 2026-09-09, issue #48: the old engine froze every IRQ in the block on one lost EOI.)

9.2.3 IOWIN Register (APB 0x004)

Bits	Name	Type	Reset	Description
[31:0]	data	RW	0x00000000	Read/Write data for register selected by IOREGSEL

Behavior: - Reading IOWIN returns data from register selected by current IOREGSEL value - Writing IOWIN updates register selected by current IOREGSEL value - IOREGSEL value persists until explicitly changed, so repeated IOWIN accesses to the same internal register need only one IOREGSEL write; write IOREGSEL again only to select a different register

9.2.4 Internal Registers (Accessed via IOREGSEL/IOWIN)

9.2.4.1 IOAPICID Register (Internal Offset 0x00)

Access via IOREGSEL/IOWIN:

```
*IOREGSEL = 0x00;  // Select IOAPICID
*IOWIN = 0x0F000000;  // Set APIC ID = 0xF
```

Bits	Name	Type	Reset	Description
[23:0]	Reserved	RO	0x000000	Reserved, read as 0

Bits	Name	Type	Reset	Description
[27:24]	APIC ID	RW	0x0	4-bit IOAPIC identifier for multi-IOAPIC systems
[31:28]	Reserved	RO	0x0	Reserved, read as 0

Purpose: Identifies this IOAPIC in systems with multiple I/O APICs.

9.2.4.2 IOAPICVER Register (Internal Offset 0x01)

Access via IOREGSEL/IOWIN:

```
*IOREGSEL = 0x01;  // Select IOAPICVER
uint32_t ver = *IOWIN;  // Read version
```

Bits	Name	Type	Reset	Description
[7:0]	Version	RO	0x11	IOAPIC version (0x11 for 82093AA compatibility)
[15:8]	Reserved	RO	0x00	Reserved, read as 0
[23:16]	Max Redir Entry	RO	0x17	Maximum redirection entry (0x17 = 23 for 24 IRQs)
[31:24]	Reserved	RO	0x00	Reserved, read as 0

Purpose: Reports IOAPIC capabilities to software.

9.2.4.3 IOAPICARB Register (Internal Offset 0x02)

Access via IOREGSEL/IOWIN:

```
*IOREGSEL = 0x02;  // Select IOAPICARB
uint32_t arb = *IOWIN;  // Read arbitration ID
```

Bits	Name	Type	Reset	Description
[23:0]	Reserved	RO	0x000000	Reserved, read as 0
[27:24]	Arbitration ID	RO	0x0	Bus arbitration priority (mirrors APIC ID)
[31:28]	Reserved	RO	0x0	Reserved, read as 0

Purpose: Multi-IOAPIC bus arbitration (read-only, matches APIC ID).

9.2.4.4 IOAPICARBCFG Register (Internal Offset 0x03)

NOT an 82093AA register. Selector 0x03 is reserved on the real part, and this block uses it for the one thing the datasheet's arbitration scheme cannot express: a choice about fairness. Software that never writes it gets the 82093AA behaviour.

Access via IOREGSEL/IOWIN:

```
*IOREGSEL = 0x03;   // Select IOAPICARBCFG
*IOWIN = 0x1;       // Round robin
```

Bits	Name	Type	Reset	Description
[0]	rr_enable	RW	0x0	0 = static priority, 1 = round robin
[31:1]	Reserved	RO	0x0	Reserved, read as 0

Static priority is the 82093AA scheme: scan up from pin 0, the lowest eligible number wins. Its weakness follows from the rule rather than from a bug – a level pin that becomes eligible again the cycle after software EOIs it holds the low ground forever, and every pin above it starves.

Round robin starts the scan just above the pin that was last ACCEPTED, and wraps. The pin just served becomes the last one the scan reaches, so every eligible pin is served before any pin is served twice, and priority becomes a position in the rotation rather than an IRQ number.

The pointer moves only on an accept. A pick the consumer never takes must not move it, or a stalled consumer would walk the rotation round the ring without delivering anything.

9.2.4.5 IOAPICMSIADDR / IOAPICMSIDATA Registers (Internal Offsets 0x04, 0x05)

NOT 82093AA registers. Selectors 0x04 and 0x05 are reserved on the real part – the same treatment as IOAPICARBCFG. Software written for the 82093AA never writes them, and at their reset value of 0 nothing observes them.

They exist because an MSI is, in the end, just a write to an address. The delivery channel already carries a message; the `ioapic_msi_emit` companion turns that message into a posted write, and these two registers hold the parts of the write the message itself does not carry. Nothing in `apb4_ioapic` consumes them: they leave the block as `cfg_msi_addr` and `cfg_msi_data`, and if the companion is not instantiated they are inert storage.

Access via IOREGSEL/IOWIN:

```
*IOREGSEL = 0x04;          // Select IOAPICMSIADDR
*IOWIN = 0xFEE00000;       // x86 local-APIC message region

*IOREGSEL = 0x05;          // Select IOAPICMSIDATA
*IOWIN = 0x00000000;       // template; the message fills in the low
bits
```

Bits	Name	Type	Reset	Description
[31:0]	addr	RW	0x00000000	IOAPICMSIADDR. Target address; [19:12] are replaced by the message destination

Bits	Name	Type	Reset	Description
[31:0]	data	RW	0x00000000	IOAPICMSIDATA. Data template; [11:0] are replaced by the message fields

What the message overwrites. The companion passes both registers through except for the bits the delivery message defines:

Field	Source
addr[19:12]	destination
addr, all other bits	IOAPICMSIADDR
data[7:0]	vector
data[10:8]	delivery mode
data[11]	destination mode
data[31:12]	IOAPICMSIDATA

That split is the x86 convention and it is a DEFAULT, not a requirement – nothing in the 82093AA dictates an MSI format. It is expressed as two programmable registers rather than parameters so the target address and the fixed data bits can be changed at runtime, which is what a driver expects.

Refusal comes back as retry. The posted write's PSLVERR is returned on the delivery channel as `deliv_retry`, so a refused MSI is re-offered by the same path that handles a LAPIC refusing a LowestPriority message. Timing stays posted: the write is issued and the response consumed when it arrives.

9.2.4.6 IOAPICMSIDROP Register (Internal Offset 0x06)

NOT an 82093AA register. Selector 0x06 is reserved on the real part, the same treatment as IOAPICARBCFG and the MSI address/data pair.

Read-only, hardware-owned, saturating. It counts refused deliveries that could NOT be acted on – `irq_out_retry` asserting while no delivery handshake is in progress.

Bits	Name	Type	Reset	Description
[31:0]	count	RO	0x00000000	Saturating count of refusals that arrived too late to act on

What it does and does not count. A consumer that refuses DURING the handshake is re-offered by the core and is not counted: nothing was lost. `ioapic_lowest_pri_arb` is combinational, so its refusals land in that category. A refusal that arrives afterwards cannot be acted on, because the core qualifies retry at the handshake – by then the edge has been retired as accepted and the interrupt is gone.

That is exactly the posted-MSI case: `ioapic_msi_emit` derives `deliv_ready` from the master's `cmd_ready`, so the handshake closes when the write is QUEUED, while PSLVERR comes back later – always too late. Such an MSI is dropped. This register is what makes that visible rather than silent.

It **saturates** at 0xFFFFFFFF rather than wrapping. A wrapped count reading 3 after four billion drops is worse than one pinned at its maximum, because the small number looks like good news.

9.2.4.7 IOAPICBOOTINTX Register (Internal Offset 0x07)

NOT an 82093AA register. Selector 0x07 is reserved on the real part, the same treatment as IOAPICARBCFG, the MSI pair and IOAPICMSIDROP.

Bits	Name	Type	Reset	Description
[0]	enable	RW	0x0	1 = a masked pin also drives its mapped legacy PIC input
[31:1]	Reserved	RO	0x0	Reserved, read as 0

Boot interrupt is the chipset behaviour where a device's INTx is rerouted to the legacy 8259 while the IOAPIC is not delivering it, so an interrupt raised before the OS has programmed the IOAPIC is not lost. It is a compatibility crutch, which is why it is a control rather than always-on: an OS that has programmed the IOAPIC generally wants it off, and the only other way to stop a masked pin leaking to the PIC would be to unmask it.

A pin reroutes only when this bit is set AND that pin's IOREDTBL mask bit is set. Both terms matter. Without the mask term a pin the IOAPIC is actively delivering would also reach the PIC and the interrupt would be taken twice; without the enable term the crutch could never be switched off.

The rerouting itself is not in this block. `apb4_ioapic` exports the per-pin mask as `cfg_mask_vec` and this bit as `cfg_boot_intx_en`; the `ioapic_boot_intx` companion combines them with the INTx lines and produces the 8259-bound term. An integrator that does not want boot interrupts simply does not instantiate it, and with the enable at its reset value of 0 nothing reroutes even if they do.

9.2.5 Redirection Table (Internal Offsets 0x10-0x3F)

Each of the 24 IRQ inputs has a 64-bit redirection entry consisting of two 32-bit registers (LO and HI).

Internal Offset Calculation:

```
// For IRQ n (0-23):
uint8_t offset_lo = 0x10 + (n * 2);      // Even offset
uint8_t offset_hi = 0x10 + (n * 2) + 1;  // Odd offset
```

```
// Examples:
// IRQ0:  LO=0x10, HI=0x11
// IRQ1:  LO=0x12, HI=0x13
// IRQ23: LO=0x3E, HI=0x3F
```

9.2.5.1 IOREDTBL[n]_LO Register (Lower 32 Bits)

Access via IOREGSEL/IOWIN:

```
*IOREGSEL = 0x10 + (irq_num * 2);  // Select LO register for IRQ n
*IOWIN = config_value;              // Write config
```

Bits	Name	Type	Reset	Description
[7:0]	Vector	RW	0x00	Interrupt vector to deliver to CPU (0x00-0xFF)
[10:8]	Delivery Mode	RW	0b000	000=Fixed, 001=LowestPri, 010=SMI, 100=NMI, 101=INIT, 111=ExtINT
[11]	Dest Mode	RW	0	Destination mode: 0=Physical, 1=Logical
[12]	Delivery Status	RO	0	0=Idle, 1=Send Pending (read-only)
[13]	Polarity	RW	0	Interrupt polarity: 0=Active High, 1=Active Low
[14]	Remote IRR	RO	0	Remote IRR: 0=No IRQ, 1=IRQ accepted (read-only,

Bits	Name	Type	Reset	Description
				level mode only)
[15]	Trigger Mode	RW	0	Trigger mode: 0=Edge, 1=Level
[16]	Mask	RW	1	Interrupt mask: 0=Not Masked (enabled), 1=Masked (disabled)
[31:17]	Reserved	RO	0x0000	Reserved, read as 0

Field Descriptions:

Vector [7:0]: - Interrupt vector number delivered to CPU - Typically 0x20-0xFF for x86 systems (0x00-0x1F reserved) - Software chooses unique vector per IRQ or shares vectors

Delivery Mode [10:8]: - **000 (Fixed):** Deliver to CPU specified in Destination field (MVP) - **001 (Lowest Priority):** Deliver to the lowest-priority CPU in the destination set. DELEGATED: the IOAPIC forwards the mode and the destination set and the local APICs arbitrate among themselves, as they did on the APIC bus. The IOAPIC's half is `irq_out_retry`, which lets the receiver say nobody could accept so the interrupt is offered again - **010 (SMI):** System Management Interrupt (future) - **100 (NMI):** Non-Maskable Interrupt (future) - **101 (INIT):** Initialization sequence (future) - **111 (ExtINT):** External Interrupt, 8259-compatible (future)

Destination Mode [11]: - **0 (Physical):** Use Destination field as physical APIC ID (MVP) - **1 (Logical):** Use Destination field as logical destination (future)

Delivery Status [12] (Read-Only): - **0 (Idle):** This IRQ is not in the delivery stage - **1 (Send Pending):** This IRQ is presented on `irq_out_valid` and not yet accepted. It drops on the accept; a level interrupt waiting for its EOI shows in Remote IRR, not here - Hardware-controlled, at most one IRQ reports Send Pending at a time

Polarity [13]: - **0 (Active High):** IRQ asserted when input is high - **1 (Active Low):** IRQ asserted when input is low - Allows direct connection to active-low interrupt sources - Changing this bit does not fabricate an edge: the pin's edge detector is suppressed for one cycle after the polarity bit changes, so software may flip polarity on an idle pin without a spurious interrupt

Remote IRR [14] (Read-Only, Level Mode Only): - **0:** No interrupt or interrupt serviced - **1:** Level interrupt accepted by CPU, waiting for EOI - Only meaningful for level-triggered interrupts - Prevents re-triggering of this pin until EOI received; other pins are not affected - Set when the CPU

accepts the delivery (`irq_out_valid && irq_out_ready`), not when it is presented - an EOI that arrives before the accept is dropped - Cleared by an EOI whose vector equals the vector that was *delivered* for this pin (latched at accept), not the RTE's current vector field - The compare runs on every pin at once: one EOI clears Remote IRR on *every* pin whose delivered vector matches, so two pins sharing a vector both leave service on one EOI (82093AA behaviour)

Rewriting an RTE's vector while its interrupt is in flight is safe. The CPU EOIs the vector it was delivered, and that is what the clear compares against, so the EOI still lands; the next delivery of the pin uses the new vector. (Fixed 2026-09-09, issue #48: the clear used to compare against the live RTE field, which turned a mid-service rewrite into a permanently blocked pin and forced a mask-and-wait rule on software.)

Trigger Mode [15]: - **0 (Edge):** Interrupt on rising edge of active signal - **1 (Level):** Interrupt on active level, uses Remote IRR, needs EOI

Mask [16]: - **0 (Not Masked):** IRQ enabled, can generate interrupts - **1 (Masked):** IRQ disabled, no interrupts generated - Default is masked (1) for safety - Software must unmask to enable IRQ

9.2.5.2 IOREDTBL[n]_HI Register (Upper 32 Bits)

Access via IOREGSEL/IOWIN:

```
*IOREGSEL = 0x10 + (irq_num * 2) + 1;  // Select HI register for IRQ n
*IOWIN = dest_value;                    // Write destination
```

Bits	Name	Type	Reset	Description
[23:0]	Reserved	RO	0x000000	Reserved, read as 0
[31:24]	Destination	RW	0x00	Destination: Physical APIC ID or Logical destination

Field Descriptions:

Destination [31:24]: - **Physical Mode (Dest Mode=0):** 8-bit physical APIC ID of target CPU - **Logical Mode (Dest Mode=1):** Logical destination for multi-cast (future) - For single-CPU systems, typically set to CPU's APIC ID (often 0x00 or 0x01)

9.2.6 Complete Register Address Map

9.2.6.1 Internal Register Offsets (via IOREGSEL)

Offset	Register	Type	Description
System Registers			
0x00	IOAPICID	RW	I/O APIC

Offset	Register	Type	Description
			identification
0x01	IOAPICVER	RO	Version and max entry
0x02	IOAPICARB	RO	Arbitration priority
0x03	IOAPICARBCFG	RW	Arbitration policy (`rr_enable[0]`). NOT an 82093AA register
0x04	IOAPICMSIADDR	RW	MSI target address. NOT an 82093AA register
0x05	IOAPICMSIDATA	RW	MSI data-word template. NOT an 82093AA register
0x06	IOAPICMSIDROP	RO	Saturating count of refused deliveries that could not be acted on. NOT an 82093AA register
0x07	IOAPICBOOTINTX	RW	Boot-interrupt rerouting arm (`enable[0]`). NOT an 82093AA register
0x08-0x0F	Reserved	-	Unmapped: the selector stores and reads back, IOWIN access is dropped
Redirection Table			
0x10	IOREDTBL[0]_LO	RW	IRQ0 redirection

Offset	Register	Type	Description
			entry low
0x11	IOREDTBL[0]_HI	RW	IRQ0 redirection entry high
0x12	IOREDTBL[1]_LO	RW	IRQ1 redirection entry low
0x13	IOREDTBL[1]_HI	RW	IRQ1 redirection entry high
0x14	IOREDTBL[2]_LO	RW	IRQ2 redirection entry low
0x15	IOREDTBL[2]_HI	RW	IRQ2 redirection entry high
…	…	…	(continues for all 24 IRQs)
0x3E	IOREDTBL[23]_LO	RW	IRQ23 redirection entry low
0x3F	IOREDTBL[23]_HI	RW	IRQ23 redirection entry high

9.2.6.2 Typical IRQ Assignments (PC-Compatible Systems)

IRQ #	Offset (LO/HI)	Traditional Use	Typical Vector
IRQ0	0x10/0x11	System Timer	0x20
IRQ1	0x12/0x13	Keyboard	0x21
IRQ2	0x14/0x15	Cascade (PIC)	-
IRQ3	0x16/0x17	COM2	0x23
IRQ4	0x18/0x19	COM1	0x24
IRQ5	0x1A/0x1B	LPT2 / Sound	0x25
IRQ6	0x1C/0x1D	Floppy	0x26
IRQ7	0x1E/0x1F	LPT1	0x27
IRQ8	0x20/0x21	RTC Alarm	0x28
IRQ9	0x22/0x23	ACPI / SCSI	0x29
IRQ10	0x24/0x25	Available	0x2A
IRQ11	0x26/0x27	Available	0x2B

IRQ #	Offset (LO/HI)	Traditional Use	Typical Vector
IRQ12	0x28/0x29	PS/2 Mouse	0x2C
IRQ13	0x2A/0x2B	FPU	0x2D
IRQ14	0x2C/0x2D	Primary IDE	0x2E
IRQ15	0x2E/0x2F	Secondary IDE	0x2F
IRQ16-23	0x30-0x3F	PCI Interrupts, Additional devices	0x30-0x37

9.3 Usage Example

9.3.1 Example 1: Configure IRQ14 (IDE) - Edge-Triggered

```
// Configure IRQ14 for primary IDE controller
// Vector 0x2E, Edge-triggered, Active High, Fixed delivery, Unmask

// Step 1: Select IOREDTBL[14]_LO (offset 0x2C)
*IOREGSEL = 0x2C;

// Step 2: Write LO configuration
//   Bits[7:0]   = 0x2E       (Vector)
//   Bits[10:8]  = 0b000      (Fixed delivery)
//   Bit[11]     = 0          (Physical dest)
//   Bit[13]     = 0          (Active high)
//   Bit[15]     = 0          (Edge trigger)
//   Bit[16]     = 0          (Not masked)
*IOWIN = 0x0000002E;

// Step 3: Select IOREDTBL[14]_HI (offset 0x2D)
*IOREGSEL = 0x2D;

// Step 4: Write destination (CPU 0)
*IOWIN = 0x00000000;  // Destination APIC ID = 0
```

9.3.2 Example 2: Configure IRQ9 (ACPI) - Level-Triggered

```
// Configure IRQ9 for ACPI SCI (System Control Interrupt)
// Vector 0x29, Level-triggered, Active Low, Fixed delivery, Unmask

// Step 1: Select IOREDTBL[9]_LO (offset 0x22)
*IOREGSEL = 0x22;

// Step 2: Write LO configuration
//   Bits[7:0]   = 0x29       (Vector)
//   Bits[10:8]  = 0b000      (Fixed delivery)
```

```
//   Bit[11]     = 0          (Physical dest)
//   Bit[13]     = 1          (Active LOW)
//   Bit[15]     = 1          (LEVEL trigger)
//   Bit[16]     = 0          (Not masked)
*IOWIN = 0x0000A029;  // Bits 15,13 set for level, active-low

// Step 3: Select IOREDTBL[9]_HI (offset 0x23)
*IOREGSEL = 0x23;

// Step 4: Write destination (CPU 0)
*IOWIN = 0x00000000;
```

9.3.3 Example 3: Mask/Unmask an IRQ

```
// Mask IRQ5 (disable)
*IOREGSEL = 0x1A;           // Select IOREDTBL[5]_LO
uint32_t config = *IOWIN;    // Read current config
config |= (1 << 16);         // Set mask bit
*IOWIN = config;             // Write back

// Unmask IRQ5 (enable)
*IOREGSEL = 0x1A;           // Select IOREDTBL[5]_LO
config = *IOWIN;             // Read current config
config &= ~(1 << 16);        // Clear mask bit
*IOWIN = config;             // Write back
```

9.3.4 Example 4: Check Delivery Status and Remote IRR

```
// Check if IRQ12 (PS/2 mouse) is being delivered
*IOREGSEL = 0x28;           // Select IOREDTBL[12]_LO
uint32_t status = *IOWIN;
bool is_pending = (status & (1 << 12)) != 0;      // Delivery Status
bool remote_irr = (status & (1 << 14)) != 0;      // Remote IRR

if (is_pending) {
    printf("IRQ12 delivery in progress\n");
}

if (remote_irr) {
    printf("IRQ12 waiting for EOI (level-triggered)\n");
}
```

9.3.5 Example 5: Read IOAPIC Version and Capabilities

```
// Discover IOAPIC capabilities
*IOREGSEL = 0x01;           // Select IOAPICVER
uint32_t ver_reg = *IOWIN;

uint8_t version = ver_reg & 0xFF;
```

```
uint8_t max_entry = (ver_reg >> 16) & 0xFF;
uint8_t num_irqs = max_entry + 1;

printf("IOAPIC Version: 0x%02X\n", version);
printf("Number of IRQs: %d\n", num_irqs);
```

9.4 Design Notes

9.4.1 Register Field Summary

Control Fields (Software Writable):

Field	Register	Purpose
Vector[7:0]	REDIR_LO	Interrupt vector number
Delivery Mode[10:8]	REDIR_LO	How to deliver (Fixed, LowestPri, etc.)
Dest Mode[11]	REDIR_LO	Physical vs Logical addressing
Polarity[13]	REDIR_LO	Active High vs Active Low
Trigger Mode[15]	REDIR_LO	Edge vs Level
Mask[16]	REDIR_LO	Enable/Disable this IRQ
Destination[31:24]	REDIR_HI	Target CPU APIC ID
APIC ID[27:24]	IOAPICID	This IOAPIC's identifier

Status Fields (Read-Only):

Field	Register	Purpose
Delivery Status[12]	REDIR_LO	Interrupt delivery in progress
Remote IRR[14]	REDIR_LO	Level interrupt waiting for EOI
Version[7:0]	IOAPICVER	IOAPIC version (0x11)
Max Redir Entry[23:16]	IOAPICVER	Number of IRQs - 1 (0x17)
Arbitration ID[27:24]	IOAPICARB	Bus arbitration priority

9.4.2 Reset Values

After Reset: - IOREGSEL = 0x00 - IOAPICID = 0x00000000 (ID = 0) - All IOREDTBL entries: - Vector = 0x00 - Delivery Mode = 0b000 (Fixed) - Dest Mode = 0 (Physical) - Delivery Status = 0 (Idle) - Polarity = 0 (Active High) - Remote IRR = 0 - Trigger Mode = 0 (Edge) - **Mask = 1 (MASKED) ←** Important! All IRQs disabled by default - Destination = 0x00

Software must unmask IRQs to enable interrupts.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 I/O Advanced Programmable Interrupt Controller (IOAPIC)

Status: Implemented - 37/37 in all six DV configurations (2026-09-09) **Priority:** Medium **Address:** `0xFEC0_6000 - 0xFEC0_6FFF` (4KB window)

10.1 Overview

82093AA-compatible interrupt router: 24 IRQ inputs, a programmable redirection table reached through the IOREGSEL/IOWIN indirect window, and a single valid/ready delivery interface to the CPU/LAPIC.

10.2 What is implemented

- 24 IRQ inputs, asynchronous, three-stage synchronized in `ioapic_core`
- Edge and level trigger modes, active-high/active-low polarity, per-pin mask
- Arbitration in two policies: static priority (lowest IRQ number wins, the 82093AA scheme and the reset default) or round robin behind `IOAPICARBCFG.rr_enable`
- One outstanding delivery on a valid/ready handshake - no delivery FSM - with `irq_out_retry` on the same handshake, so a receiver that could not place the interrupt gets it offered again instead of losing it
- Per-pin Remote IRR: a level interrupt blocks ITS OWN pin until EOI, other pins keep delivering, and a lost or wrong-vector EOI cannot stall the block
- EOI matched against the vector actually DELIVERED on that pin, so an RTE may be re-pointed between delivery and EOI
- IOREGSEL/IOWIN indirect access. Only APB `0x000` (IOREGSEL) and `0x004` (IOWIN) are software-visible in the 4 KB window; the indirect pair is the only path to the register file. Three classes of access never reach the register block:
  - an address above `0x0FF` - dropped, PSLVERR (it would otherwise alias onto the 8-bit register address modulo `0x100`)
  - an address inside `0x000-0x0FF` that is neither IOREGSEL nor IOWIN - dropped, PSLVERR. This was a backdoor: such an address used to reach the register block at its raw offset, so a direct write to `0x008` rewrote IOAPICID and one to `0x014` rewrote IOREDTBL[0].REDIR_LO

    - IOWIN with a selector the 82093AA does not implement - dropped, and NOT an error: the address is legal, the register is not, and the architectural answer is a read of zero
- CDC_ENABLE=1: the whole CPU/LAPIC-facing interface is presented in pclk and crosses into ioapic_clk through matched-latency synchronizers

10.3 LowestPriority is delegated, and that is not a dodge

An IOAPIC does not track CPU priority, and never did. On the APIC bus it broadcast the message and the local APICs arbitrated among themselves using their Arbitration Priority Registers – roughly the maximum of a CPU's task priority class, its highest in-service vector and its highest pending one – and whichever was lowest accepted. The IOAPIC only had to know whether anybody took it.

That is why the destination, the destination mode and the delivery mode are forwarded unmodified. The half this block was missing was never the choosing. It was being told the choice FAILED.

`irq_out_retry` is that half:

handshake	meaning	what the block does
`ready` low	the receiver is not ready	nothing; the offer stands, the rotation does not move
`ready`, `retry` low	a CPU accepted	retire the edge latch, set Remote IRR, latch the delivered vector, move the rotation
`ready`, `retry` high	consumed, nobody could accept	move the rotation and OFFER IT AGAIN – nothing retires

Tie `irq_out_retry` low and the channel behaves exactly as it did before this existed: every completed handshake is an acceptance.

What a consumer owes. For delivery mode 001 the receiver gets the vector and the destination set and must pick the lowest-priority CPU in it, or refuse with retry when none can take it. None of that lives here, which is the point.

One sharp edge, stated rather than hidden. Under static priority a refused pin is still the lowest eligible number, so it wins the next arbitration immediately and a persistent refusal monopolises the channel. Round robin fixes it, which is the same answer static priority's starvation has everywhere else in this block.

Why the channel is shaped this way. The intent is to hang a CPU off it across a bus. A payload plus a handshake plus a status bridges onto AMBA readily – the retry becomes a bus response. A

bundle of live per-CPU priority registers would not: it would be stale by the time it crossed, and the IOAPIC would be choosing on numbers that had already moved.

10.4 Arbitration is static unless you say otherwise

Static priority is the 82093AA scheme: scan up from pin 0, the lowest eligible number wins. Its weakness is a consequence of the rule rather than a bug. A level pin that becomes eligible again the cycle after software EOIs it holds the low ground forever, and every pin above it starves.

Writing 1 to `IOAPICARBCFG.rr_enable` starts the scan just above the pin that was last ACCEPTED, and wraps. The pin just served becomes the last one the scan reaches, so every eligible pin is served before any pin is served twice. Priority becomes a position in the rotation rather than an IRQ number.

The pointer moves only on an accept. A pick that the consumer never takes must not move it, or a stalled consumer would walk the rotation round the ring without delivering anything.

`IOAPICARBCFG` is NOT an 82093AA register. Selector `0x03` is reserved on the part, and this block uses it for the one choice the datasheet's scheme cannot express. A driver written for the 82093AA never writes it and gets static priority.

10.5 Destination mode

`irq_out_dest_mode` carries the RTE's destination mode alongside `irq_out_dest`: 0 means the destination is a physical APIC ID, 1 means it is a logical bitmask. An IOAPIC does not decode logical destinations itself - it forwards the field and the mode, and the local APICs do the matching - so forwarding the mode is the whole of this block's responsibility for logical delivery (RLB TASK-008, was RLB-008). Delivery modes other than Fixed are likewise forwarded on `irq_out_deliv_mode` unmodified.

10.6 Feature history (RLB TASK-008 (was RLB-008), closed 2026-09-14)

Nothing here is outstanding: RLB TASK-008 (was RLB-008) closed 2026-09-14 with every feature built. The section is kept because both items it once listed as BLOCKED were wrong, and the corrections are worth keeping.

MSI SHIPPED 2026-09-14 as the `ioapic_msi_emit` companion. The old claim – "an APB slave has no initiator port" – was true and beside the point: an MSI is a posted write, so a companion bridges the delivery channel onto an `apb4_master_stub`. Its target address and data are registers (IOAPICMSIADDR/IOAPICMSIDATA, IOWIN selectors 0x04/0x05), and a refusal that arrives too late to act on is counted in IOAPICMSIDROP (0x06).

Boot interrupt was MIS-SCOPED, not blocked. The old text described INIT-SIPI-SIPI, which is a LOCAL APIC's AP-startup IPI and was never this block's business. The real feature is the chipset behaviour: a PCI device's INTx rerouted to the legacy PIC while IOAPIC delivery is masked, so an

interrupt raised before the OS programs the IOAPIC is not lost. SHIPPED as the fourth companion, `ioapic_boot_intx`, gated on IOAPICBOOTINTX.enable (IOWIN selector 0x07) AND the pin's RTE mask – without the mask term a pin the IOAPIC is actively delivering would reach the PIC too and be taken twice. `rlb_top` specifies the map: IOAPIC pin n to legacy PIC input n for pins 0-7, no reroute above.

Multi-IOAPIC routing is available as a companion: `ioapic_deliv_merge` merges N delivery channels onto one and tags each message with its source id, so an EOI can be routed back to the IOAPIC holding that pin's Remote IRR.

LowestPriority is delegated rather than absent, and the other delivery modes (SMI, NMI, INIT, ExtINT) are FORWARDED rather than acted on: they ride `irq_out_deliv_mode` unmodified and it is the receiver that interprets them. See "LowestPriority is delegated" above for what the block does and does not owe in that arrangement.

10.7 Files

File	Role
`apb4_ioapic.sv`	Top level: APB slave (CDC or not), LAPIC interface crossing
`ioapic_core.sv`	Synchronization, edge/level tracking, arbitration, Remote IRR
`ioapic_config_regs.sv`	IOREGSEL/IOWIN translation, PeakRDL wrapper, hwif mapping
`ioapic_regs.sv`, `ioapic_regs_pkg.sv`	PeakRDL generated - regenerate only via `bin/peakrdl_generate.py`
`../../rdl/ioapic/ioapic_regs.rdl`	Register source of truth (fixed at 24 entries)
`filelists/apb4_ioapic.f`	Compile closure

10.8 Verification

`projects/components/legacy-ip/retro_legacy_blocks/dv/tests/test_apb4_ioapic.py`, six configurations (CDC off/on x gate/func/full). The defect-regression suite for GitHub issue #48 is `dv/tbclasses/ioapic/ioapic_tests_medium.py`.

```
cd projects/components/legacy-ip/retro_legacy_blocks/dv/tests
make clean-all && make run-apb4_ioapic-full
```

10.9 Specification

`projects/components/legacy-ip/retro_legacy_blocks/docs/ioapic_mas/`

Last Updated: 2026-09-09

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

11 RLB (Retro Legacy Blocks) - FPGA Implementation Guide

Date: 2025-11-16 **Purpose:** Explain RLB blocks and FPGA implementation compatibility

---

11.1 What Are RLB (Retro Legacy Blocks)?

11.1.1 Overview

Retro Legacy Blocks are modern implementations of classic PC/embedded system peripherals, designed as reusable IP cores with APB interfaces. They provide functionality compatible with legacy systems but implemented in clean, modern SystemVerilog.

11.1.2 Purpose

Educational: - Learn how classic PC peripherals work internally - Understand interrupt controllers, timers, power management - Study register-based peripheral design - Practice verification of complex protocols

Practical: - Build soft-core SoC systems on FPGAs - Create retro computer replicas (PC-compatible) - Implement embedded systems with familiar peripherals - Rapid prototyping of control systems

Industrial: - Reusable IP cores for custom SoCs - Known-good peripheral implementations - Well-documented, tested components - Modern coding standards with legacy compatibility

---

11.2 Implemented RLB Modules

11.2.1 Complete Modules

11.2.1.1 1. HPET (High Precision Event Timer)

- **Purpose:** High-resolution timer (sub-microsecond precision)
- **Features:** Multiple timers, one-shot/periodic modes, 64-bit counters
- **PC Equivalent:** Modern replacement for PIT
- **Use Cases:** Precise timing, event scheduling, benchmarking

11.2.1.2 2. PIT_8254 (Programmable Interval Timer)

- **Purpose:** Classic PC timer (3 channels)
- **Features:** Mode 0-5 operation, binary/BCD counting
- **PC Equivalent:** 8254 PIT

- **Use Cases:** System tick, speaker control, legacy PC compatibility

11.2.1.3 3. RTC (Real-Time Clock)

- **Purpose:** Calendar and alarm functions
- **Features:** Time/date tracking, alarm, periodic interrupts
- **PC Equivalent:** MC146818 / DS12887
- **Use Cases:** Wall-clock time, wake-on-alarm, timestamping

11.2.1.4 4. PIC_8259 (Programmable Interrupt Controller)

- **Purpose:** Legacy interrupt management
- **Features:** 8 IRQ inputs, priority, cascading, masking
- **PC Equivalent:** 8259A PIC
- **Use Cases:** Legacy PC interrupts, simple systems

11.2.1.5 5. SMBus (System Management Bus)

- **Purpose:** Low-speed system management communication
- **Features:** I2C/SMBus 2.0, master/slave, PEC, FIFOs
- **PC Equivalent:** Modern motherboard SMBus controller
- **Use Cases:** Sensor monitoring, battery management, system config

11.2.1.6 6. PM_ACPI (Power Management ACPI)

- **Purpose:** Advanced power management
- **Features:** PM timer, power states (S0/S1/S3), GPE, clock gating, power domains
- **PC Equivalent:** ACPI PM controller
- **Use Cases:** Low-power systems, battery devices, sleep/wake, clock control

11.2.1.7 7. IOAPIC (I/O Advanced Programmable Interrupt Controller)

- **Purpose:** Advanced interrupt routing
- **Features:** 24 IRQs, redirection table, edge/level, priority, EOI
- **PC Equivalent:** 82093AA I/O APIC
- **Use Cases:** Multi-processor systems, flexible IRQ routing, modern PCs

---

11.3 FPGA Implementation - Nexys A7 / Genesys2

11.3.1 Can RLB Be Implemented on These Boards? YES!

All RLB modules are **100% compatible** with Nexys A7 and Genesys2 FPGAs.

11.3.2 Target FPGA Specifications

11.3.2.1 Nexys A7 (Artix-7)

- **FPGA:** XC7A100T-1CSG324C (or XC7A50T)
- **Logic Cells:** 101,440 (A100T) or 52,160 (A50T)
- **Block RAM:** 4,860 Kb
- **Clock:** 100 MHz oscillator
- **I/O:** Extensive (PMOD, switches, LEDs, UART, etc.)
- **Verdict: Excellent fit for full RLB system**

11.3.2.2 Genesys 2 (Kintex-7)

- **FPGA:** XC7K325T-2FFG900C
- **Logic Cells:** 326,080
- **Block RAM:** 16,020 Kb
- **Clock:** 200 MHz oscillator
- **I/O:** Very extensive
- **Verdict: More than sufficient, ideal for complex SoC**

11.3.3 Resource Usage Estimates

Per Module (Approximate):

Module	LUTs	FFs	BRAM	Notes
HPET	~800-1200	~600-900	0	Multi-timer, 64-bit counters
PIT_8254	~400-600	~300-500	0	3 channels, modes
RTC	~300-500	~250-400	0	Time/date logic
PIC_8259	~200-400	~150-300	0	Simple priority logic
SMBus	~600-900	~500-700	0.5	FIFOs, protocol FSM
PM_ACPI	~500-800	~400-600	0	Power FSM,

Module	LUTs	FFs	BRAM	Notes
IOAPIC	~800-1200	~600-900	0	GPE 24 IRQs, redirection

Total RLB System (~5500 LUTs, ~4300 FFs): - **Nexys A7-100T:** ~5% logic utilization Plenty of room - **Nexys A7-50T:** ~11% logic utilization Still comfortable - **Genesys 2:** ~2% logic utilization Massive headroom

11.3.4 What Can Fit on These Boards?

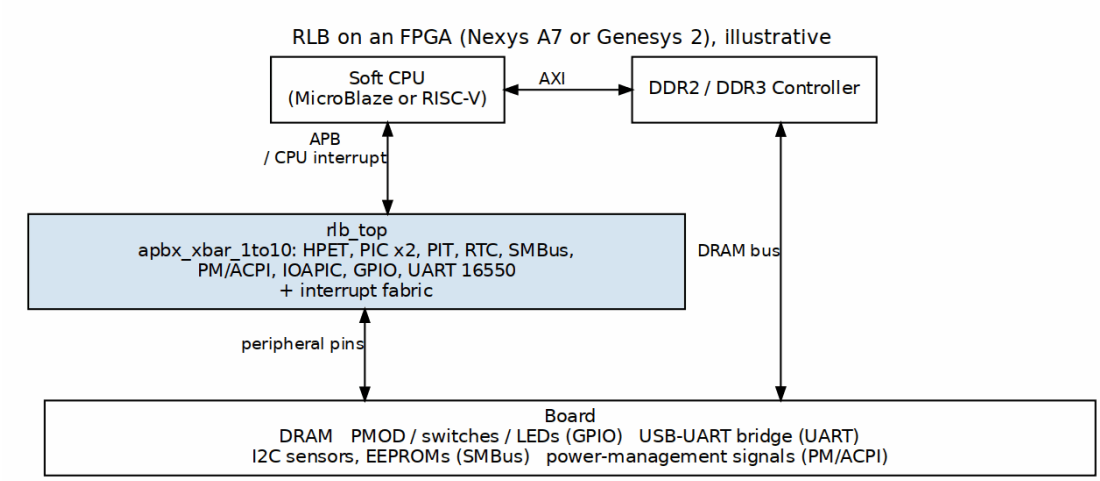
Nexys A7-100T Can Support: - All 7 RLB modules - MicroBlaze or RISC-V soft processor - DDR2 controller - Ethernet MAC - VGA/HDMI display controller - USB interface - Audio codec - **Result:** Complete retro-PC or modern embedded system

Genesys 2 Can Support: - Everything above PLUS: - Multiple soft processors - PCIe endpoint - High-speed interfaces - Complex video processing - **Result:** Professional-grade SoC development platform

---

11.4 RLB System Integration on FPGA

11.4.1 Typical FPGA System Architecture



RLB on an FPGA, illustrative

11.4.2 Example Use Cases on FPGAs

11.4.2.1 1. Retro PC Replica

Build a PC-compatible system on FPGA: - Soft x86 core (or compatible RISC-V with x86 emulation) - All RLB peripherals for PC compatibility - VGA output for display - PS/2 keyboard/mouse - Run DOS, early Windows, Linux

11.4.2.2 2. Embedded Control System

Industrial/automotive control: - RISC-V processor - HPET for precise timing - GPIO for sensors/actuators - SMBus for temperature/voltage monitoring - PM_ACPI for power optimization - UART for debug/communication

11.4.2.3 3. Education Platform

Learn computer architecture: - Study peripheral internals - Practice interrupt handling - Understand power management - Experiment with different configurations - Debug with real hardware

11.4.2.4 4. SoC Prototyping

Rapid prototyping: - Test peripheral configurations - Validate interrupt routing - Optimize power consumption - Benchmark performance - Hardware/software co-development

---

11.5 Nexys A7 Specific Integration

11.5.1 Available Resources on Nexys A7

Peripherals Already on Board: - 16 switches (GPIO inputs) - 16 LEDs (GPIO outputs) - 5 push buttons - 7-segment displays (4 digits) - USB-UART bridge - PMOD connectors - VGA output - Microphone - Speaker - Temperature sensor - Accelerometer

11.5.2 How RLB Modules Map to Nexys A7

RLB Module	Nexys A7 Resource	Connection
GPIO	Switches, LEDs, Buttons	Direct mapping
UART_16550	USB-UART bridge	FT2232HQ chip
HPET	Internal timing	No external pins needed
PIT_8254	Internal timing	Timer/speaker
RTC	Internal clock	Optional ext crystal

RLB Module	Nexys A7 Resource	Connection
PIC/IOAPIC	Internal routing	IRQ aggregation
SMBus	I2C PMOD	Via I2C PMOD adapter
PM_ACPI	Power control	Clock gates, power logic

11.5.3 Example Nexys A7 Top-Level

```
module nexys_a7_rlb_system (
    // Clock and reset
    input  wire         clk_100mhz,    // 100 MHz oscillator
    input  wire         cpu_resetn,    // Reset button

    // GPIO
    input  wire [15:0]  sw,            // Switches → GPIO inputs
    output wire [15:0]  led,           // LEDs ← GPIO outputs
    input  wire [4:0]   btn,           // Buttons → GPIO/interrupts

    // UART
    input  wire         uart_rxd,      // USB-UART RX
    output wire         uart_txd,      // USB-UART TX

    // I2C/SMBus (via PMOD)
    inout  wire         smbus_sda,
    inout  wire         smbus_scl,

    // 7-segment display
    output wire [6:0]   seg,
    output wire [7:0]   an,

    // VGA (optional)
    output wire [3:0]   vga_r, vga_g, vga_b,
    output wire         vga_hs, vga_vs
);

    // Instantiate soft processor (MicroBlaze, RISC-V, etc.)
    // Instantiate APB crossbar
    // Instantiate all RLB modules
    // Connect to board resources

endmodule
```

11.6 Implementation Recommendations

11.6.1 For Nexys A7-100T

Recommended RLB Configuration:

```
HPET        - System timing
PIT_8254    - Legacy timer for compatibility
RTC         - Real-time clock
PIC_8259    - Simple interrupt controller (OR IOAPIC)
GPIO        - Board I/O (switches, LEDs, buttons)
UART_16550  - Serial communication
SMBus       - I2C peripherals
PM_ACPI     - Optional (if power features needed)
```

Resource Budget: - RLB modules: ~6,000 LUTs, ~5,000 FFs - MicroBlaze: ~2,000-3,000 LUTs - DDR2 Controller: ~2,000 LUTs - VGA/Display: ~1,000 LUTs - **Total:** ~11,000-12,000 LUTs (11% of XC7A100T)

Plenty of room for: - User applications - Additional peripherals - Debug logic - Custom IP cores

11.6.2 For Genesys 2

Can Support Everything: - All RLB modules simultaneously - Multiple soft processors - Complex memory hierarchy - High-speed peripherals - Video processing - Network interfaces

Recommended for: - Professional SoC development - Multi-core systems - High-performance applications - Complex retro systems (full PC replica)

11.7 FPGA-Specific Considerations

11.7.1 Clock Management

Nexys A7: - 100 MHz main clock → Can generate all needed frequencies - Use MMCM/PLL for: - CPU clock (50-200 MHz) - Peripheral clocks (25-100 MHz) - Display clocks (25.175 MHz for VGA) - USB clock (60 MHz)

RLB Modules: Most work at system clock (50-100 MHz), some support CDC for different clocks

11.7.2 Power Management

PM_ACPI Use on FPGA: - Clock gating controls → Xilin x clock enables - Power domains → Can't control FPGA power, but can simulate - Useful for: Power-aware designs, testing, learning ACPI - Can control external power via GPIO pins

11.7.3 Interrupt Routing

IOAPIC vs PIC_8259 Choice: - **Simple systems (single CPU):** Use PIC_8259 (smaller) - **Advanced systems:** Use IOAPIC (more flexible, PC-compatible) - **Maximum flexibility:** Include both, use jumpers/config

11.7.4 External Interfaces

How to Connect RLB to Board Resources:

1. **GPIO Module:**
   - Inputs: Switches, buttons
   - Outputs: LEDs, 7-segment displays
   - Bidirectional: PMOD I/O pins
2. **UART_16550:**
   - Connect to FT2232HQ USB-UART
   - Baud rate generator uses system clock
   - CTS/RTS for flow control (optional)
3. **SMBus:**
   - Connect to I2C PMOD
   - On-board: Temperature sensor (ADT7420)
   - On-board: Accelerometer (ADXL362)
   - External: I2C EEPROMs, sensors via PMOD
4. **Timers (HPET/PIT):**
   - Internal only, no external pins
   - Can drive speaker output
   - Can generate PWM via GPIO
5. **RTC:**
   - Internal timekeeping
   - Optional: External 32.768 kHz crystal
   - Battery backup simulation

11.8 Soft Processor Options

11.8.1 MicroBlaze (Xilinx)

- **Pros:** Well-supported, debugger, IP integration
- **Cons:** Proprietary, Vivado-specific

- **RLB Compatibility:** Excellent (APB via AXI interconnect)

11.8.2 RISC-V (Open Source)

- **Options:** VexRiscv, PicoRV32, Rocket, BOOM
- **Pros:** Open-source, portable, modern ISA
- **Cons:** Need to build toolchain
- **RLB Compatibility:** Excellent (APB native or via AXI)

11.8.3 Custom Soft Core

- Can design custom CPU specifically for RLB peripherals
- Full control over instruction set
- Educational value

11.9 Step-by-Step FPGA Implementation

11.9.1 Phase 1: Basic System (Week 1-2)

1. **Create FPGA project** (Vivado for Xilinx)
2. **Add simple processor** (MicroBlaze or RISC-V)
3. **Add APB crossbar** components/fabric-gen-ip/lattice-apbx-xbar/
4. **Add one RLB module** (start with GPIO)
5. **Connect to LEDs/switches**
6. **Test basic register access**

11.9.2 Phase 2: Expand Peripherals (Week 3-4)

7. **Add UART_16550** → Test serial communication
8. **Add HPET** → Test precise timing
9. **Add PIC_8259 or IOAPIC** → Test interrupts
10. **Add RTC** → Test time keeping

11.9.3 Phase 3: Advanced Features (Week 5-6)

11. **Add SMBus** → Test I2C communication
12. **Add PM_ACPI** → Test power management
13. **Add all remaining** RLB modules
14. **System integration testing**

11.9.4 Phase 4: Application Development (Week 7+)

15. **Develop firmware** using RLB peripherals
16. **Test real-world applications**
17. **Optimize and debug**
18. **Hardware/software co-verification**

11.10 Example Address Map for FPGA

```
Base Address | Module      | Size | Description
-------------|-------------|------|------------------------
0xFEC0_0000  | HPET        | 4KB  | High Precision Timer
0xFEC0_1000  | PIC_8259    | 4KB  | Interrupt Controller (master)
0xFEC0_2000  | PIT_8254    | 4KB  | Programmable Interval Timer
0xFEC0_3000  | RTC         | 4KB  | Real-Time Clock
0xFEC0_4000  | SMBus       | 4KB  | System Management Bus
0xFEC0_5000  | PM_ACPI     | 4KB  | Power Management
0xFEC0_6000  | IOAPIC      | 4KB  | I/O APIC
0xFEC0_7000  | GPIO        | 4KB  | General Purpose I/O
0xFEC0_8000  | UART_16550  | 4KB  | Serial Port
0xFEC0_9000  | PIC_8259    | 4KB  | Interrupt Controller (cascade
slave)
```

All accessible via APB from soft processor. The base is the `BASE_ADDR` parameter of `rlb_top` (default `0xFEC00000`); the window order is the generated `apbx_xbar_1to10`'s and is not free to rearrange.

11.11 Software Development

11.11.1 Bare-Metal Firmware

Using RLB Modules in C:

```c
#include "rlb_peripherals.h"

// Initialize system
void system_init(void) {
    // Configure GPIO
    gpio_set_direction(0xFFFF0000);  // Upper 16 = outputs

    // Initialize UART
    uart_init(115200);
    uart_puts("System starting...\n");
```

```
    // Configure HPET timer
    hpet_init();
    hpet_start_timer(0, 1000000);  // 1ms periodic

    // Setup interrupts via IOAPIC
    ioapic_configure_irq(14, 0x2E, IRQ_EDGE, 0);  // IDE
    ioapic_configure_irq(15, 0x2F, IRQ_EDGE, 0);  // IDE

    // Enable power management
    pm_acpi_init();
}
```

11.11.2 Operating System Support

Can Run: - **Bare-metal:** Direct hardware access - **RTOS:** FreeRTOS, Zephyr with RLB drivers - **Linux:** With proper device drivers for RLB - **DOS/DOS-compatible:** If using x86-compatible core - **Custom OS:** Educational OS development

---

11.12 Advantages of RLB on FPGA

11.12.1 Development Benefits

1. **Rapid Prototyping:** Change peripheral config instantly (no PCB)
2. **Debugging:** ChipScope/ILA for internal signals
3. **Flexibility:** Easy to add/remove peripherals
4. **Cost-Effective:** One board, infinite configurations
5. **Educational:** See everything, change everything

11.12.2 Technical Benefits

1. **Known-Good IP:** Well-documented, tested peripherals
2. **Standard Interfaces:** APB everywhere, easy integration
3. **Scalable:** Start simple, add complexity
4. **Portable:** Pure SystemVerilog, vendor-independent
5. **Modern Design:** Clean code, proper methodology

11.12.3 Vs. Microcontroller

Why FPGA + RLB instead of MCU?

Feature	MCU	FPGA + RLB
Peripherals	Fixed	Infinite flexibility
Interrupts	Fixed routing	Programmable (IOAPIC)
Timers	Limited count	As many as needed
Clock domains	1-2	Unlimited with CDC
Custom logic	Limited	Full flexibility
Learning value	Black box	Full visibility
Cost	Lower	Higher but more capable

11.13 Conclusion

11.13.1 Can RLB Be Used on Nexys A7/Genesys2? ABSOLUTELY!

Perfect Match Because: - Plenty of logic resources - Rich I/O for peripheral connections - Clock resources for multi-domain design - Development boards with extensive support - Educational and professional use

Recommended Starting Point: 1. Nexys A7-100T for most projects 2. Genesys 2 for complex/multi-core systems 3. Start with GPIO + UART + HPET 4. Add peripherals incrementally 5. Full RLB system fits comfortably with room to spare

The RLB modules are specifically designed to be FPGA-friendly and work excellently on these development boards!

Last Updated: 2025-11-16 **Recommendation:** Nexys A7-100T is ideal for complete RLB-based SoC development

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 APB HPET Micro-Architecture Specification

Component: APB High Precision Event Timer (HPET) **Version:** 1.4 **Last Updated:** 2026-10-13 **Status:** RTL Functional - register interface validated; HPET_STATUS per-bit W1C with a reset value and a core-owned status mirror, independent counter halves, strobe-driven comparator loads and the no-re-fire armed latch are as specified (fixed 2026-09-08, issue #46); periodic catch-up (missed periods skipped, never burst; period 1 steps to counter + 1), the halted-counter-only comparator re-arm and the next-epoch hold (an advance that carries out of the compare width is held off until the counter wraps there) are as specified (review rounds 1 and 2, 2026-09-09)

12.1 Overview

This MAS describes the APB HPET as the RTL exists today. The register-side defects that issue #46 collected – a W1C that cleared every bit on any write, status storage with no reset, a counter load that shipped the previous write's other half, and completed one-shots re-firing on every enable – were fixed on 2026-09-08, and two review rounds the next day tightened the rules the chapters below now state: a comparator write re-arms only a stopped timer (so a half-written 64-bit comparator can never fire on the torn value), a periodic timer that falls behind catches up silently instead of firing once and going quiet (at period 1 by stepping to counter + 1), and a comparator whose advance carries out of the compare width is held in the counter's next epoch until the counter wraps there, rather than matching early. Legacy replacement is no longer one of those limitations: RLB/hpet TASK-003 implemented the LegacyReplacement Route, so `leg_rt_cap` reads 1 and timers 0/1 move to IRQ0/IRQ8 when it is enabled. The intentional limitations that remain (`timer_value_set` is storage with nothing behind it, and general per-timer I/O APIC routing is unimplemented so `TIMER_INT_ROUTE_CAP` reads 0) are called out in-line where you'll meet them. The spec is organized into five chapters covering the micro-architecture of the APB HPET component.

Table: Version History

Version	Date	Author	Changes
1.0	2026-01-04	RTL Design Sherpa	Initial MAS release
1.1	2026-09-08	RTL Design Sherpa	Issue #46 fixes: per-bit W1C status mirror with reset, independent counter halves, strobe-driven comparator loads, armed-latch fire (no re-fire on enable),

Version	Date	Author	Changes
			HPET_ID from parameters
1.2	2026-09-09	RTL Design Sherpa	Issue #46 review: comparator writes re-arm only a stopped timer (torn 64-bit value cannot fire; disable-write-enable contract), periodic catch-up skips missed periods without bursting, period 0/1 behaviour, same-cycle write-and-fire rule, halted-counter rule for byte-strobed counter writes
1.3	2026-09-09	RTL Design Sherpa	Issue #46 review round 2: period-1 catch-up steps to counter + 1 and terminates on advance >= counter; next-epoch hold bit (`r_comp_next_epoch`) for an advance that carries out of the compare width, with its four clears and the width consequences; stopped-timer comparator write always re-arms (a value at or below the

Version	Date	Author	Changes
			counter fires on enable, no wait for wrap); running-timer reprogramming consequences stated as outside the contract; width-masked advance in 32-bit mode
1.4	2026-10-13	RTL Design Sherpa	Stale pre-spec references swept out: comparator offsets +0x08/+0x0C in every example table, waveform and memory-map diagram; global offsets (CONFIG 0x010, STATUS 0x020, COUNTER 0x0F0/0x0F4); the retired per-timer `timer_enable` removed from port tables, FSM transitions and code excerpts (the only stop is `hpet_enable`); HPET_STATUS is set on every fire, `timer_int_enable` gates only the interrupt; TIMER_CONFIG reads 0x30 out of

Version	Date	Author	Changes
			reset; reserved offsets read 0 without PSLVERR

12.2 Navigation

12.2.1 Chapter 1: Overview

- 01_overview.md - Component overview, features, applications
- 02_architecture.md - High-level architecture and block hierarchy
- 03_clocks_and_reset.md - Clock domains and reset behavior
- 04_acronyms.md - Acronyms and terminology
- 05_references.md - External references and standards

12.2.2 Chapter 2: Blocks

- 00_overview.md - Block hierarchy overview
- 01_hpet_core.md - Core timer logic (counter, comparators, FSM)
- 02_hpet_config_regs.md - Configuration register wrapper
- 03_hpet_regs.md - PeakRDL generated register file
- 04_apb4_hpet_top.md - Top-level integration
- 05_fsm_summary.md - FSM state summary table

12.2.3 Chapter 3: Interfaces

(planned, not yet written - see `../../rtl/hpet/apb4_hpet.sv` for the current port list) - 01_top_level.md - Top-level signal list - 02_apb_interface_spec.md - APB protocol specification - 03_hpet_clock_interface.md - HPET clock domain interface - 04_interrupt_interface.md - Timer interrupt outputs

12.2.4 Chapter 4: Programming Model

(planned, not yet written - see the HPET section of ../../CLAUDE.md for programming notes) - 01_initialization.md - Software initialization sequence - 02_timer_configuration.md - Configuring timers (one-shot, periodic) - 03_interrupt_handling.md - Interrupt service routines - 04_use_cases.md - Common use case examples

12.2.5 Chapter 5: Registers

- 01_register_map.md - Complete register address map and field descriptions